



Bachelor Degree Project

Stronger Together? An Ensemble of CNNs for Deepfakes Detection



Author: Angelica Gardner
Supervisor: Tobias Ohlsson
Semester: VT 2020
Subject: Computer Science

Abstract

Deepfakes technology is a face swap technique that enables anyone to replace faces in a video, with highly realistic results. Despite its usefulness, if used maliciously, this technique can have a significant impact on society, for instance, through the spreading of fake news or cyberbullying. This makes the ability of deepfakes detection a problem of utmost importance. In this paper, I tackle the problem of deepfakes detection by identifying deepfakes forgeries in video sequences. Inspired by the state-of-the-art, I study the ensembling of different machine learning solutions built on convolutional neural networks (CNNs) and use these models as objects for comparison between ensemble and single model performances. Existing work in the research field of deepfakes detection suggests that escalated challenges posed by modern deepfake videos make it increasingly difficult for detection methods. I evaluate that claim by testing the detection performance of four single CNN models as well as six stacked ensembles on three modern deepfakes datasets. I compare various ensemble approaches to combine single models and in what way their predictions should be incorporated into the ensemble output. The results I found was that the best approach for deepfakes detection is to create an ensemble, though, the ensemble approach plays a crucial role in the detection performance. The final proposed solution is an ensemble of all available single models which use the concept of soft (weighted) voting to combine its base-learners' predictions. Results show that this proposed solution significantly improved deepfakes detection performance and substantially outperformed all single models.

Keywords: deepfakes, deepfakes detection, supervised learning, binary classification, convolutional neural networks, ensemble learning, stacking

*This is how you win ML competitions: you take other peoples' work and
ensemble them together.*

- Vitaly Kuznetsov, NIPS 2014

Preface

There is a statement circulating in variations on the Internet that goes a little something like this: “I would like to thank Stack Overflow for this degree.” This statement is mostly regarded as a laugh, but I believe it highlights the hard work of those who came before us. I’m indebted to all preceding research where the authors have made their work public and open source for followers like me to use their ideas, findings, source code, and datasets.

In addition, this thesis would not have happened without the support of several people who deserve thanks:

First and foremost are my family, I can not forget the love, patience, support, engagement, and sacrifices of my husband, Najib, and children. My isolation was endured and overlooked by the people I love the most. The fact that God gave me such a family is something that deserves eternal gratitude.

I would like to show my appreciation to my parents and grandparents. They were early intellectual inspirations in my life, giving me the ability to think outside the box and explore new ideas. Their love, energy, and support have shaped who I am and anything I say can never truly express the gratitude that is due to them. I pray that God gives them long, healthy lives full of happiness and love.

I want to acknowledge and thank my supervisor, Tobias, for helping me with valuable insights, writing suggestions, and encouragement along the way. This also includes all classmates who gave beneficial feedback and interesting comments on the content of this report.

Finally, this degree project would not have been possible without Linnaeus University: the up-to-date education they offer in the fields of Computer Science, and the great engagement of their teachers.

Stockholm, 31st of May 2020. Angelica Gardner

Contents

1 Introduction	7
1.1 Background	8
1.1.1 The deepfake technology	8
1.1.2 Training machine learning models	9
1.1.3 Artificial neural networks	11
1.1.4 Convolutional neural networks	14
1.1.5 Ensemble learning	17
1.1.6 Evaluating machine learning models	19
1.2 Related work	21
1.2.1 Capsule	23
1.2.2 DSP-FWA	24
1.2.3 Ictu Oculi	25
1.2.4 XceptionNet	26
1.3 Problem formulation	26
1.4 Motivation	27
1.5 Objectives	28
1.6 Scope	29
1.7 Target group	29
1.7.1 Deepfakes detection community	30
1.7.2 Programmers	30
1.7.3 Social networking companies	30
1.8 Outline	30
2 Method	31
2.1 Datasets and data preprocessing	31
2.2 Experimental setup	33
2.2.1 Training setup	33
2.2.2 Ensemble learning setup	34
2.2.3 Evaluation setup	35
2.3 Reliability and Validity	36
2.4 Ethical Considerations	37
3 Implementation	39
3.1 Environmental setup	39
3.2 Collecting and preprocessing data	40

3.3 Implementing, training, and evaluating single CNNs	42
3.4 Creating and evaluating ensembles	43
4 Results	44
4.1 Single model performances	45
4.2 Ensemble performance	50
5 Analysis	54
5.1 Single model performances	55
5.2 Ensemble performance	58
6 Discussion	60
7 Conclusion	62
7.1 Future work	63
References	64

1 Introduction

The *deepfake* phenomenon has been a prominent discussion topic in recent years. Deepfakes are videos where a face swap technique replaces the face of a target individual with the face of another person while the remaining background scene and the original facial expressions are preserved, as seen in Figure 1.1. Deepfake technology is part of deep learning where machine learning (ML) models based on artificial neural networks learn to detect and classify data representations [1]. In this context, the data represents human faces and since faces symbolize identity, a well-crafted deepfake can create the illusion of an individual’s behavior that did not occur in reality, making it look like this person speaks and performs in ways he/she never did.



Figure 1.1: Image from an original video (left) and another from a fake video produced using deepfake technology (right). The videos are part of the Celeb-DF dataset [14].

In response to this phenomenon gaining attraction, detection methods have been introduced to identify forged images and videos created by deepfake technology [2]. Detection approaches vary; some strategies [3], [4] build on smart contracts that trace the history of the image or video in order to determine its originality and authenticity, while other strategies [5], [6], [7], [8] use machine learning models to classify videos as being real or fake. As for the latter, the methods differ with regards to the architecture, choice of algorithm, and configurations.

One type of deep learning method that has shown noteworthy progress in the fields of computer vision and image processing is convolutional neural networks. These neural networks have demonstrated exemplary performance in ML competitions and are recognized as state-of-the-art in vision-related applications [9], including deepfakes detection [2]. Even though many of these detection methods display promising performances, there is still a concern that deepfake technology continues to evolve, even utilizing the latest detection methods to its advantage, resulting in new generations of fake videos that gradually become more difficult to discover for current models [2], [10]. Consequently, the interest to advance these detection methods also

continues, and therefore, this project aims to investigate how the process of ensemble learning can be utilized to improve deepfakes detection.

Ensemble learning is an established way to improve the stability and accuracy of ML algorithms by creating a collection of models working together. This collection of models is called an *ensemble* and is commonly used to enhance overall performance [1]. Studies in related research fields show how ensembles of multiple models demonstrate better results than single models, and in various public ML competitions, winning solutions have been ensemble methods [11].

Accordingly, the hypothesis for this research is that developing an ensemble for deepfakes detection will produce a robust model with a more accurate detection performance than what single models can achieve. To establish this, it's necessary to evaluate how different deepfakes detection models perform on recent generations of deepfake videos and then build upon these models to develop the ensemble. As mentioned, convolutional neural networks have shown particular success in similar fields and for that reason, such models will be the main focus of this research.

1.1 Background

The purpose of this section is to briefly describe deepfake technology and how it relates to machine learning, introduce artificial neural networks including the specific type of convolutional neural network, provide a quick review of ensemble learning, and finally, explain some relevant aspects to the process of training and evaluating machine learning models.

1.1.1 The deepfake technology

The beginning of the deepfake technology is attributed to an unidentified user on the social media platform Reddit¹ in November 2017. In December that same year, the user's source code was uploaded to GitHub² (one of the leading code sharing platforms) for the purpose of giving the developer community an opportunity to collaborate and further develop the idea [12]. Since then, deepfake technology has evolved and made it possible to produce fake videos of better and more trustworthy quality. The phenomenon has spread additionally by the community introducing similar projects and even applications for users without coding skills.

The core idea behind the deepfake technology lies in using generative adversarial networks (GANs). GAN is a class of ML systems where the networks consist of two components called autoencoders: the *generator* and the *discriminator* [11]. The creation of a deepfake video starts with an input video of a target individual and the generator is trained to create imagery

¹ <https://www.reddit.com/>

² <https://github.com/deepfakes/faceswap>

where the target's face is replaced by that of another person. The discriminator is then trained to classify if the imagery is real or if it's coming from the generator. These two autoencoders work in a feedback loop where the generator understands its mistakes from the data it gets from the discriminator, whereby it can improve its process to create more realistic synthetic representations [13]. The resulting fake videos can achieve a high level of realism, however, as with other ML models, if a GAN has a bad configuration or isn't trained properly, the deepfake technology will produce visual flaws and inconsistencies that can be more or less obvious. These defects include flickering facial areas, color mismatches, a lack of reasonable eye blinking, incoherent head poses, blurred areas, and other poorly reconstructed details. An example can be seen in Figure 1.2.



Figure 1.2: Image from a real video (left) and two images from deepfake videos (middle and right). There is an obvious difference in quality between the two deepfakes, where the image to the right has apparent visual flaws.

The videos are part of the Celeb-DF dataset [14].

As briefly mentioned in the introduction section, ML models can be trained to detect errors and deviations in fake videos that differentiate them from authentic ones. This has been confirmed to be a successful approach for deepfakes detection [2], yet, research has also shown that detection performance weakens considerably for newer generations of deepfakes with better visual quality [14].

A large part of detection models is based on artificial neural networks, particularly its subclass convolutional neural network [2]. Therefore, it's appropriate to briefly review these types of algorithms and how they can be used for this research purpose, but first I'll describe some aspects about training ML models that are relevant to the experiment conducted in this study.

1.1.2 Training machine learning models

In this study, the experiment will apply *supervised learning* as the training approach for the algorithms. Supervised learning means that the ML model is given a training set of labeled data where each input $\{x_1, x_2, \dots, x_n\}$ is

associated with a label that identifies which category the input belongs to $\{y_1, y_2, \dots, y_n\}$, this is known as a classification problem [1]. In supervised learning, we have known mappings between the inputs and the desired outputs which enables us to check how accurate the model is at its predictions. The problem of deepfakes detection is a classic example of a binary classification problem as the goal is to predict whether a video is real or fake (i.e. assigning the input to one of two predefined classes).

During training, the model goes through multiple iterations over a training dataset containing data samples used for learning how to find optimal configurations for accurate predictions. In the course of the training phase, a problem that is known as *overfitting* commonly appears. Overfitting (also called high variance) happens when the model starts to pick up random noise from the data that doesn't represent true patterns [1]. This leads to the model memorizing the training data rather than learning how to make correct classifications. A model that experiences high variance will be unable to apply what it has learned to new and previously unseen data and predictions from that model will be unreliable, this is referred to as the model's generalization capability [15].

The opposite of overfitting is known as *underfitting*. Underfitting (also called high bias) happens when the model was not able to learn enough patterns from the data during training, reasonably by missing relevant relationships between input and output [1]. This problem will also lead to poor generalization capability and unreliable predictions [15]. The essence of overfitting and underfitting can be seen in Figure 1.3.

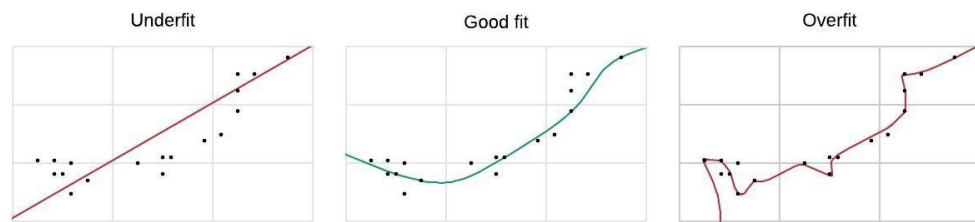


Figure 1.3: When a model is underfitting, it has not learned enough patterns from the data and when it's overfitting, it has memorized the data.

A performance that balances overfitting and underfitting is desirable, and this is described as the *bias-variance tradeoff* [15]. There exist various techniques to find this balance and one common method is to split the training data into one subset for training and one subset for validation [1]. During training, the training set is used as usual for learning while the validation set is used to test if the model is learning the right patterns. If the model keeps getting predictions wrong on the validation set, we know something is wrong in the learning process. Through validation, we also get information about how

accurate the model is on training data as well as on validation data, and if we notice that training error is less than validation error, we can understand the model is starting to overfit by memorizing the training data [15]. On such occasions, a technique called *early stopping* can be used to stop the training process and in that way prevent high variance. The technique of early stopping is part of a group of strategies known collectively as *regularisation* [1]. These strategies deal with the central problem of how to make an ML model perform well on new inputs, not just on the training data, thereby improving the model's generalization capability. Other configuration options for regularisation and training settings, particularly associated with artificial neural networks, are explained in the next section.

1.1.3 Artificial neural networks

Artificial neural network (ANN) is an approach in ML where the model structure is built up by layers of artificial neurons: building blocks referred to as *neurons* due to the fact that they are loosely inspired by neuroscience [1].

In its simplest form, an ANN is made up of three layers: an input layer that receives the initial input to the model, a hidden layer that processes the information, and an output layer that makes a decision. Artificial neurons that reside within each layer work like mathematical functions performing some action on the data before forwarding its output to the next layer. Each neuron in a given layer is connected to all or some neurons in the following layer and it's these connections that allow for the neurons to send information to each other. With every connection, there's an associated *weight* that represents the strength of this relationship between the two neurons. The neurons will increase or decrease the strength of their connection with experience which will, in turn, contribute to the network's learning [16].

There exist several categories of ANNs and the classic type is a feedforward neural network where information goes through the network in one direction [1]. ANNs learn through a gradual training process as described in [Section 1.1.2](#) where data training samples specify the output for the neural network, but the behavior of each layer is not specified. Instead, the learning algorithm has to decide in what way it must use each of its layers to obtain the desired output [16].

When training feedforward neural networks, it requires making decisions about training settings, which functions to use, and in what format the output should be [1]. As the training process for ANNs is iterative, it means we need to decide about how many times the model will run through the entire training dataset, referred to as *epochs* [16]. The number of epochs is set to an integer value between one and infinity: either the model runs for a fixed number of epochs, or you can let it run continuously until it fails to improve for a specified number of epochs.

Another relevant configuration is *batch size*. Batch size is an integer value representing the number of data samples the model will go through during training before it improves its configurations [16]. A full batch size would be the same as the number of data samples in the training set, and a mini-batch size would be anything larger than one but less than the full batch size. In practice, it has been observed that using larger batch sizes might lead to a decline in the model's generalization capability while mini-batch sizes usually lead to improved accuracy [17].

During different stages of the training process, the ANN will utilize various types of functions for specific purposes. This structure can be compiled into the five sequential actions explained below, and a visual representation is seen in Figure 1.4.

1. Every neuron in a network's layers will receive an input that it multiplies with a set of parameters: a *weight* and a *bias*. At first, the values for weight and bias are initialized to random values. For feedforward neural networks, it is important to initialize weights to small random values and biases to zero or small positive values [1]. The idea is that as the network is trained, optimal values for these parameters will be found through experience.
2. A summation of the input, weight, and bias values is passed to an *activation function* that is attached to each neuron in the network. The activation function serves as a decision operation and is used to transform the weighted sum into a single output which determines whether the neuron should be activated or not [1]. There are different types of activation functions: ReLU and sigmoid are names of two common ones used for the hidden layers, while the output layer often uses a softmax function.
3. Either before or after the activation function has been applied, a procedure referred to as *batch normalization* can be utilized to support the learning process by normalizing the model's internal representation of the data [1]. Although the effect of batch normalization is apparent, not all ANN use this technique.
4. After the input has gone through the hidden layers, the network will output a prediction and at this stage, it will seek to calculate the errors it has made in its predictions. When determining its mistakes, a *loss function* is used. The value of this loss function gives a measure of how far from perfect the performance of the ANN is. A high loss means a bad performance so the function's task is to minimize this loss (error) [16]. There are different types of loss functions, a common one is a cross-entropy loss which is often used when the activation function in the output layer is softmax [11].
5. For an ANN to learn and evolve, it has to correct these errors it found and it's at this stage the concept of *backpropagation* becomes a

central part of how ANNs acquire experience. With the concept of backpropagation, the information about prediction mistakes is transmitted in reverse (backward) through the network's layers so it can alter and adjust its configurations in the direction of less error. The configurations are modified using an *optimization function* [1]. As with the previous functions, one of several types of optimization functions is chosen: some popular ones are stochastic gradient descent (SGD) and Adam.

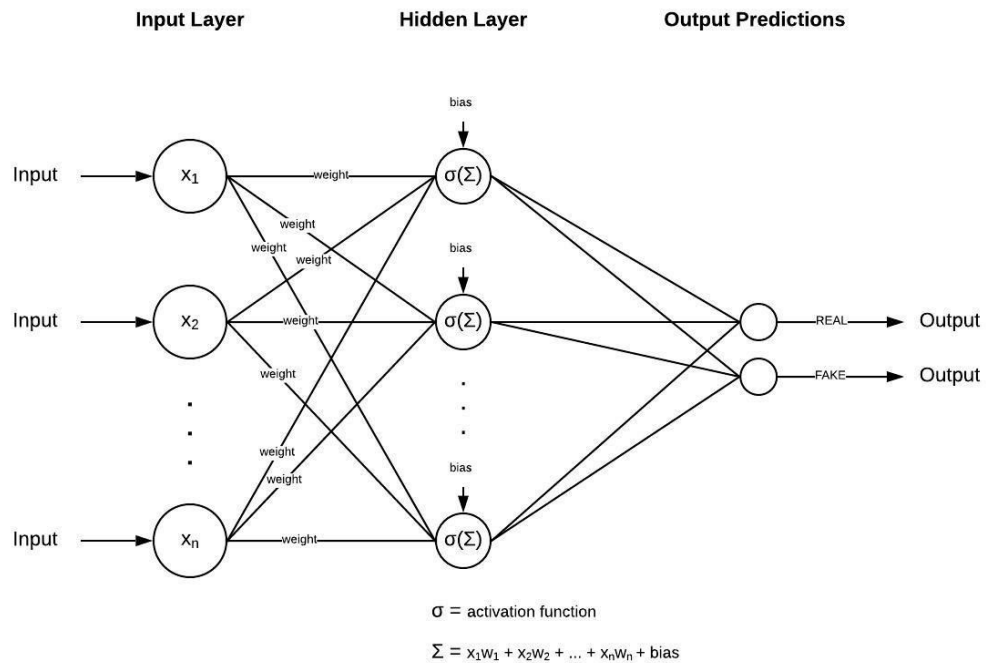


Figure 1.4: Simple structure of a feedforward ANN for binary classification.

The selected optimization function accepts configurations used to tune the optimization process and one of these is the *learning rate*. The learning rate metaphorically represents the speed at which an ML model learns and the value is often between 0.0 and 1.0 [16]. A low learning rate is considered more reliable, however, the optimization process takes a long time because steps taken towards the highest accuracy are smaller. On the contrary, a high learning rate can make the error worse by causing the model to go forward too quickly, making it diverge to insignificant solutions. Nevertheless, when an appropriate high learning rate is found, it means less time to train the model while still achieving high accuracy. Smith [18] argued that an appropriate learning rate can be estimated by training an ML model with an initially low learning rate and then increasing it with each epoch. The opposite could also be applied: starting with a large initial value and

decreasing it with each epoch. This is referred to as the *learning rate decay* and it's a decimal value that defines the way a learning rate changes over each epoch.

ANNs are highly flexible in how they process information and this has contributed to the existence of many variations of neural networks designed to solve different types of problems. The convolutional neural network is one type of ANN proven to produce strong results in the fields of computer vision and image classification [9]. Convolutional neural networks will be introduced next.

1.1.4 Convolutional neural networks

Convolutional neural network (CNN) is a feedforward ANN widely used in vision-related applications [9] that got its name from applying at least one so-called *convolutional layer* [1]. A convolutional layer divides a given input into smaller parts, uses a *kernel* to perform a convolution operation that extracts feature patterns from each part, and finally outputs a feature map. Two primary configurations we can modify to change the behavior of convolutional layers are *kernel size* and *padding*. The kernel size refers to the dimensions of the kernel, it can be of a larger size such as 11x11, a smaller size such as 3x3, or anything in between. 3x3 is currently the most widely used kernel size [19]. Padding refers to a technique where we add an additional layer of zeros as a border around an input. When the kernel is sliding over the input, corner parts will get coverage once while the middle parts get coverage numerous times. Essentially, this means we will gather more information about those middle parts, and as the size of the input volume decreases, information from corner parts might get lost [19]. To ensure we preserve as much information as possible about the original input, we can apply a padding of zeros around the input so if information from the corner parts is lost, it is merely padding.

As the generated features from a convolutional layer can be quite large, these types of layers are frequently followed by a *pooling layer* with the purpose of reducing complexity by down-sampling the convolutional output using a pooling function like average pooling or max-pooling [20]. This process of convolutional and pooling layers can be seen in Figure 1.5.

In CNNs, this base of convolutional and pooling layers is followed by a classification-part generally consisting of a mix of fully-connected layers and dropout layers. A dropout layer has a specific function in neural networks related to preventing the problem of overfitting. The layer “drops out” a random set of neurons, causing the neural network to make sure it can provide correct classifications without relying too much on specific activated neurons, preventing it from completely memorizing the training data [21].

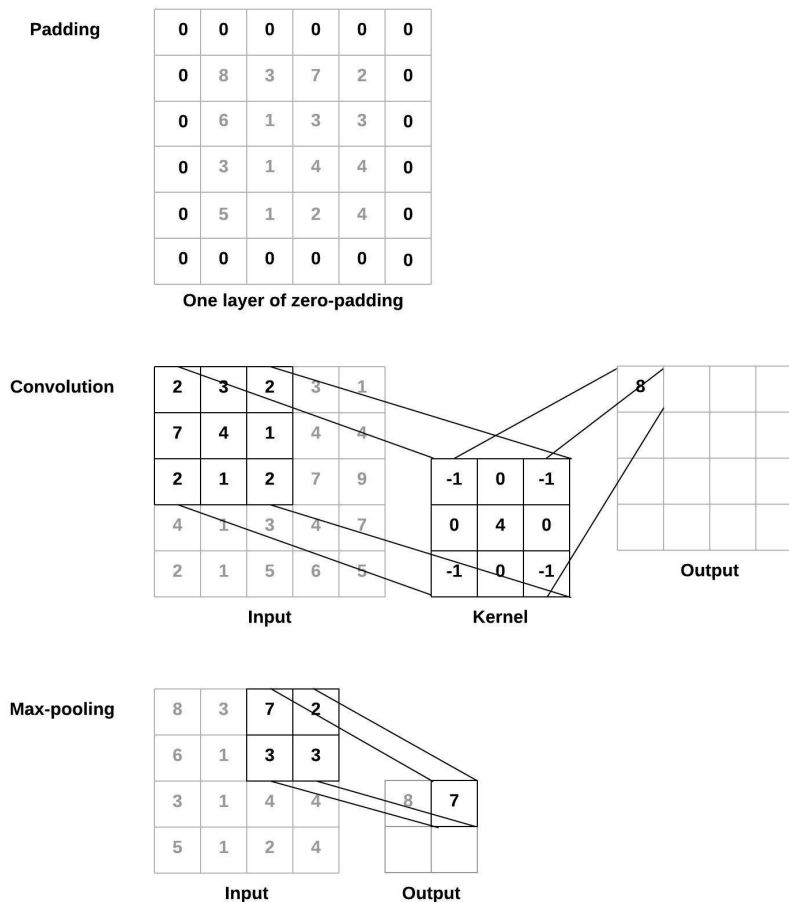


Figure 1.5: Example of a convolution process followed by a max-pooling operation where its output is the maximum value of the input.

The popularity of CNNs is primarily due to its feature extraction ability which causes the model to discover features at different levels [9]. Basically, a CNN trained on images of human faces learns to recognize lower-level features such as lines, edges, corners, circles, and other basic shapes, as well as higher-level features such as the nose, eyes, lips, and other face parts. Next, the CNN model will use the discovered features to decide on classification [1]. The process for video classification is much the same since videos essentially are many image frames composed into a complete moving picture [2].

As with other ML techniques, efforts have been carried out to improve the performance of CNNs and find solutions to their problems. Since the arrangement of CNN components proved to play a central role in achieving enhanced performance, an evolution of CNN architectures began to rise [9]. The word architecture refers to the overall structure of the network, for instance, how many layers it has and how these layers are connected to each

other [1]. Depending on the type of architectural structure, CNNs can be broadly classified into seven categories, each containing its distinct architectures [9]. In this study, I will create ensembles of multiple single CNNs used for deepfakes detection, and these models are based on different CNN architectures each belonging to one of these categories. For this reason, I will provide some details about the architectures relevant to this study: VGGNet, ResNet, and Xception.

The **VGGNet** architecture [22] is part of the category: *Spatial exploitation based CNNs*. The architectures from this category come from research where spatial filters have been exploited to improve performance and investigate their relationship with how CNNs learn [9]. In their research, Simonyan et al. [22] suggested that small size filters can improve the performance of CNNs, and thus, their proposed network model VGG19 replaced the common 11x11, 7x7, and 5x5 filters with a stack of 3x3 filters. Additionally, the convolutional layers utilize the ReLU activation function and are followed by max-pooling layers to maintain the spatial resolution. In fact, these small size filters showed to provide a good result for both image classification and localization (i.e. finding where a certain object is) while still delivering simplicity and increased network depth [9]. These findings set a new trend to work with smaller size filters. In VGGNet, the output layer uses the softmax function. VGG16 and VGG19 are common 16 and 19 layers deep CNN models that implement this architecture. The main limitations associated with VGGNet are a large number of parameters (138 million) which makes the training process slow and the resulting models impracticable to deploy because they're computationally expensive.

The **ResNet** architecture [23] is part of two categories: Depth based CNNs and Multi-Path based CNNs. For the first category, depth-based CNN architectures are based on the assumption that increased network depth plays an essential role in the classification success rate [9]. In their research, He et al. [23] constructed an efficient methodology that allowed for training very deep networks and empirically showed how their suggested architecture ResNet improved image recognition and localization tasks while still requiring less computational complexity than previously proposed networks. As for the second category, multi-path based CNN architectures address common problems CNNs face during training, such as the gradient diminishing problem, through the concept of multi-path or cross-layer connectivity. The ResNet architecture exploits this idea by systematically connecting one layer to another, allowing for a specialized flow of information across layers. Combined, ResNet revolutionized the CNN architectural race by introducing its concept of residual learning and suggesting substantially deep model variations with 50, 101, and 152 layers [9]. In summary, this architecture is characterized by its depth while still being substantially smaller in model size than previous CNN architectures.

The convolutional layers use kernel sizes of 7x7 and 3x3, and each convolutional layer is followed by batch normalization and ReLU activation function. The pooling layers use global average pooling, and the output layer utilizes the softmax function.

The **Xception** architecture [24] is part of the category: Width-based multi-connection CNNs. This architecture utilizes a different form of convolution process in its convolutional layers. Xception exploits the idea of depthwise separable convolutional layers which is a variant of the traditional convolutional layer, but these types of layers split the kernel into two separate kernels for doing two convolutions instead of one, with the intention of improving computational performance. The Xception model then transforms the convolved output as many times as the width of the layer, in contrast to only one transformation like with the conventional CNN architectures. For its pooling layers, the architecture uses max-pooling as well as global average pooling, and the final output goes through softmax before the model makes its prediction. Xception architecture is known to make learning more efficient and providing improved performance results [9].

Despite the great performance CNNs have shown, neural networks still regularly suffer from problems. The collection of regularisation techniques developed to try and prevent these issues is efficient but not a guarantee and there still exists a need to ensure improved accuracy [25]. Consequently, the concept of neural network ensembles arises. Neural network ensembling is a learning paradigm where multiple neural networks are combined to solve a problem. It originates from research by Hansen and Salamon [26] which showed that ensembling multiple ANNs and combining their predictions can help prevent overfitting and significantly improve the generalization capability of the neural network system. Accordingly, in the next section, I will introduce the concept of ensemble learning and how it can be utilized in relation to this study.

1.1.5 Ensemble learning

A collection of several ML models combined together to solve a single problem is called an *ensemble* and the method used to combine them is referred to as *ensemble learning* [1]. Zhou [27] defines the process of ensemble learning to consist of two phases: first, multiple single ML models, for example, ANNs, are trained to solve the same problem, these models are referred to as *base-learners*. Next, the trained base-learners make predictions on new and previously unseen data samples. Then, these predictions are joined together for the ensemble to make a final classification based on the base-learners predictions. This way, every member of the ensemble makes a contribution to the final output, enabling the individual weaknesses of each model to be canceled out by contributions from the other models [25]. The

concept of ensemble learning is about learning how to best combine these predictions from base-learners. Supervised learning, such as the binary classification of deepfakes detection, is a prominent situation for the application of ensemble solutions [27]. Additionally, ensemble learning is the leading winning strategy in ML competitions and often the technique used for solving real-life problems [1]. Ensembles may be as small as three, five, or ten trained base-learners but can, in theory, be as large as needed. At present, the current standard is to combine all available ML models to constitute the ensemble as ensembles tend to increase in their error correction capability the more members it has [28].

The most prevailing method approaches to create ensembles are bagging, boosting, and stacking. Bagging and boosting generally focus on creating ensembles from homogeneous base-learners, i.e. models of the same type, while stacking is the method approach to use when combining heterogeneous base-learners [27]. Typically, the main goal of the bagging method is for the ensemble model to achieve lower variance than its single members, whereas the main goal of the boosting and stacking methods is to try and produce an ensemble model with a strong classification capability [28]. Considering the single CNNs in this study will be different and that the purpose is to create a stronger model, the stacking ensemble method is the most suitable approach for this experiment.

The basic process of stacking, illustrated in Figure 1.7, is to train the single models (base-learners) on the same training set in parallel, then get every single model's prediction on a test set and use these predictions as input for the combiner ensemble. The ensemble learns how to best combine all predictions by taking every single prediction output from the base-learners as a training instance for itself where it learns how to best map the base-learners' predictions to provide an improved output. When using the stacking ensemble approach for classification problems, a process referred to as *voting* is used to combine the base-learners' predictions, and this process has two variants: hard (majority) voting or soft (weighted) voting [27]. Hard voting is when every model makes a prediction (vote) for each data sample in the test set and the final output from the ensemble is the prediction that receives more than half of the votes, i.e. the majority. If none of the predictions gets more than half of the votes, the prediction with the most votes (even if it's less than half) is selected, however, the output classification will be considered unstable for that sample. The other variant is soft voting where, at first, the training accuracy is counted for each base-learner, and then, the base-learners who showed the best performance during training will receive increased importance when the ensemble decides about its prediction.

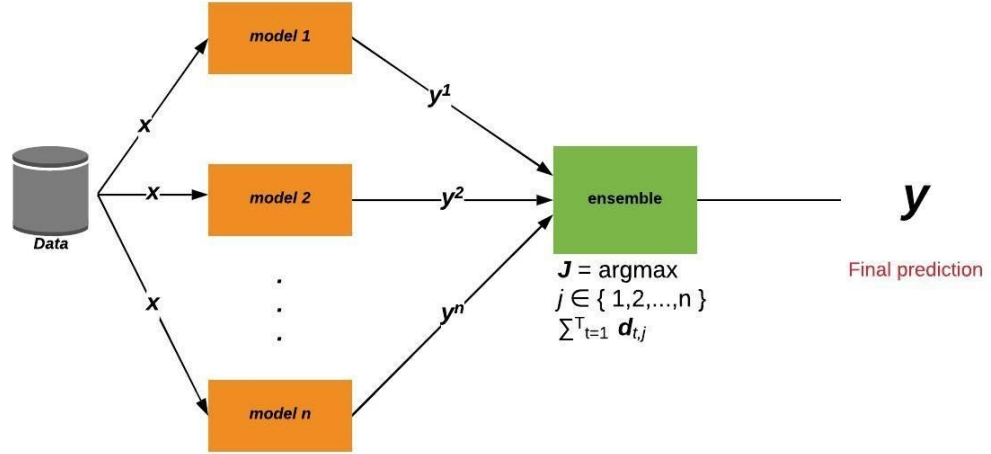


Figure 1.7: An illustration of the stacking ensemble approach.

An important concept in ensemble learning is ensemble diversity. Obviously, single models of an ensemble should be accurate, but they should also be different in some sense as ensembles tend to yield better results when there is significant diversity among its base-learners [25]. Diversity in ensembles is an effective strategy to make sure not all base-learners create the same errors, leading to an increased generalization capability of the ensemble [26]. Ensemble diversity can be achieved in several ways, among those is to combine different types of models, use various options for configurations, or to train the single models on separate datasets.

Up until now, I have described the multiple stages of developing and training single CNN models and how to combine them into ensembles. Lastly, I will present how the performance of these ML models is evaluated.

1.1.6 Evaluating machine learning models

Once an ML model is trained, it can be used to make predictions on new data by placing each example into one of the predefined classes. As mentioned in [Section 1.1.2](#), splitting the dataset into three subsets for training, validation, and testing is one way to assure these predictions are reliable. Furthermore, there exist various evaluation metrics used during testing to measure the performance quality of a model. Some common ones for classification problems include accuracy, confusion matrix, sensitivity and specificity, log loss, and area under the ROC curve (AUC) [29].

Model accuracy can be defined as the ratio between the number of correctly classified predictions and the total number of predictions as seen in Equation 1:

$$accuracy = \frac{\# \text{ correct predictions}}{\# \text{ total predictions}} \quad (1)$$

Even though we strive for highly-accurate models, accuracy alone may not be sufficient to ensure good performance as this evaluation metric makes no distinction between the different classes and therefore, we will not know if mistakes appear because the model fails to detect deepfakes or if the model is actually incorrectly classifying real videos as deepfakes [30]. In this case, looking at a confusion matrix of the results will show a more detailed breakdown of correct and incorrect predictions for each class [29]. In the context of this study, a confusion matrix would be divided into four parts as seen in Table 1.1.

True Positive (TP) Image/video is fake Prediction: deepfake	False Positive (FP) Image/video is real Prediction: deepfake
False Negative (FN) Image/video is fake Prediction: real	True Negative (TN) Image/video is real Prediction: real

Table 1.1: A binary classification confusion matrix visualizes the performance of an ML model.

Sensitivity and specificity are used in binary classification to measure the performance of a model by indicating how valid its test result is [29]. These measures use different parts of the confusion matrix for its calculations. Sensitivity, called the *true positive rate* (TPR) or *probability of detection*, measures the percentage of deepfakes that are correctly identified as fake, and as seen in Equation 2, this is calculated by dividing the number of true positives (TP) by the sum of true positives (TP) and false negatives (FN).

$$sensitivity = \frac{TP}{TP+FN} \quad (2)$$

Specificity, also called *true negative rate* (TNR), estimates the proportion of actual negatives that are correctly identified as such, e.g. the percentage of real videos correctly classified as real. As Equation 3 shows, specificity is calculated by dividing the true negatives (TN) by the sum of true negatives (TN) and false positives (FP):

$$specificity = \frac{TN}{TN+FP} \quad (3)$$

If a test result shows high sensitivity and low specificity, the model has a

high deepfakes detection rate at the same time as it also incorrectly classified many real videos as deepfakes. On the contrary, if a test shows low sensitivity and high specificity, the model fails to detect many deepfakes by incorrectly classifying them as real yet also rarely classifies real videos as fake. In the context of this study, both high sensitivity and high specificity would be desirable, however, in general, these two measures often show an anticorrelation relationship on tests [30]. Consequently, it can be argued that high sensitivity is more important for a deepfakes detection model than high specificity. Presumably, it would be better to detect more deepfakes, even if some real videos are flagged, than failing to detect deepfakes.

In [Section 1.1.3](#), the loss function was mentioned as part of how ANNs calculate its errors and measure the quality of its predictions. The lesser loss value the function produces, the higher degree of correct predictions, which makes this value a useful metric for both estimating model performance and comparing different models [30]. The loss value takes into account how certain a model is at its prediction being correct. If the prediction diverges from the actual classification, the model penalizes itself for being confident while still being wrong, and increases its loss value based on how much its prediction score varied from the correct class. Likewise, the loss value can show indications of when a model is overfitting by the training loss decreasing until the point where it's lower than the validation loss [1].

AUC stands for the *area under the ROC curve*. A ROC curve illustrates the probability of detection by laying out the rate of TP to the rate of FP [29]. As a ROC curve provides nuanced details about a model's behavior, it can be complicated to compare several ROC curves to each other and, therefore, the AUC is used as a way to summarise the performance into a single number to be compared easily [30]. The value of AUC tells us how much a model is capable of distinguishing between the different classes. AUC will always be between 0.0 and 1.0, the higher the AUC, the better the model is at predicting real as real and deepfakes as deepfakes. Hence, a low AUC is undesirable to the point that no practical model should have an AUC of less than 0.5 which would give a value worse than random guessing in the binary case [29].

1.2 Related work

In this section, I review previous work in the research field of deepfakes detection where the suggested solutions are ML models. A complete overview of all published methods would not be possible because of the rapid development in the field. Instead, this section will concentrate on the single models to be used in the experiment.

While conducting the literature review for this study, sources have, as much as possible, been selected from research papers published in academic journals and prominent conferences within the fields of machine learning,

deep learning, computer vision, and image processing. My search strategy was dependent on distinguished keywords related to the research question: *Will a neural network ensemble outperform single CNNs in deepfakes detection performance?* This research question can be separated into two components: one focusing on ensemble vs single model performance, and the other concentrating on deepfakes detection solutions. As advised by [31], a search strategy combining identified component keywords with relevant terms and synonyms, considering both singular and plural forms as well as variant spellings, is frequently the most beneficial strategy to find relevant sources that have made an impact on the research field.

Regarding the first component of the research question, keywords included ensemble, ensemble learning, convolutional neural network (CNN), neural network, and deep neural network (DNN). These searches led to finding research papers in related fields proposing neural network ensemble solutions that demonstrated more accurate estimations than single neural networks for melanoma (image) classification [11], image segmentation [28], image-based particle matter monitoring [32], data mining-based computing and computer vision for urban perception [33], among others. In [11], 135 single models and 10 ensembles of different sizes were evaluated. For some ensembles, only the best-performing single models were combined (like in this study), and for other ensembles, base-learners were selected at random. They used soft (weighted) voting to combine base-learner predictions for all ensembles. In [32], one neural network ensemble was created from all available single models using soft (weighted) voting. In [33], all available single models were combined into three ensembles using three different ensemble approaches. In [34], four single models were evaluated and compared to ensemble performance combining two, three, or all single models. They used an approach of averaging base-learners' scores to compute the final ensemble prediction.

As for the second component of the research question, keywords such as those mentioned above were combined together with terms and expressions related to deepfakes, deepfakes detection, deepfake, deep fake, and face manipulation. These searches led to research by Bonettini et al. [34] on video face manipulation detection through an ensemble of CNNs. In that study, the authors ensembled trained CNN models through two different concepts, combining these networks for detecting face manipulation in videos. The results demonstrated promising ensemble detection performance, which is encouraging for this experiment. There are various differences between this study and their research: they use CNN models not specifically developed for deepfakes detection (which I aim to), they utilize different datasets (with only one corresponding to the datasets used in this experiment), and their focus is on a solution universal enough to find forgeries of any face manipulation technique, not only deepfake technology. I have not found other research

proposing ensemble solutions for deepfakes detection or face manipulation.

The search strategy led me to multiple proposed deepfakes detection solutions based on single ANNs, particularly CNNs. To select what models to use for this experiment, I compiled four reasons to justify my selections:

- The suggested solution should be part of the research efforts made in this field during the last two years preceding this study.
- The proposed model must have shown promising deepfakes detection performance before.
- The source code must be publicly available to utilize for further studies. I find it to be out of the scope and timeframe for this study to reverse engineer any detection solutions and, therefore, only select models with available source code.
- The hypothesis and algorithmic approach for each model is distinct from others selected, ensuring diversity among them.

In this section, each model will receive a short presentation that covers its architecture, when it was published, the theory behind its research, and what performance it has shown previously - with a summary in Table 1.2. From these presentations, it should be acknowledged that all single models demonstrated high accuracy at deepfakes detection in their accompanying research papers but later, in a study by Li et al. [14], they produced a noticeable poorer performance for detecting deepfakes with higher visual quality. Their research shows how various deepfakes detection models were struggling to identify fake videos from newer and more modern datasets, proving the difficulty of deepfakes detection still remains.

Method	CNN architecture	Repository on GitHub ³	Published
Capsule [5]	VGG19	nii-yamagishilab/ Capsule-Forensics-v2	2019.10
DSP-FWA [6]	ResNet50	danmohaha/DSP-FWA	2018.11
Ictu Oculi [7]	VGG16	danmohaha/ WIFS2018_In_Ictu_Oculi	2018.06
XceptionNet [8]	Xception	ondyari/FaceForensics	2019.01

Table 1.2: A short overview of the detection models used in this study.

1.2.1 Capsule

The Capsule method [5] originates from a research paper published in 2019, suggesting to combine a CNN with a Capsule Neural Network for deepfakes

³ <https://github.com>

detection. Out of the single models selected for this experiment, the Capsule model is the newest one, it was published about seven months before this study was conducted. Capsule Neural Network (CapsNet) was introduced by Hinton et al. [35] as an ANN coming to solve some limitations of CNNs. The idea is that instead of using many succeeding layers that transfer information about the presence of object parts from one layer to the next (as with CNNs), CapsNet will apply capsules with nested convolutional layers enabling it to include information about the structural relationship between these parts, such as individual arrangements. Furthermore, the idea implies that using capsules will ensure the final result includes all important regions detected by the capsules, even if one capsule fails to detect manipulations in any region from the input, thanks to the other capsules, this region will not be missed and the final result will still be correct.

The Capsule model starts by accepting video frames as input and pass them through a VGG19 CNN before each frame enters the CapsNet. The Capsule research proposes the usage of a VGG19-model built of VGGNet [22] with 19 layers as a feature extractor to reduce high variance (overfitting). The final result is calculated by averaging the scores of all video frames.

In its original research, the Capsule model was evaluated on binary classification accuracy during two experiments where it received 94.47% and 97.69% [5]. In the research from Li et al. [14], this model was tested on both older and newer deepfakes datasets and evaluated on ROC/AUC performance. On older datasets, it achieved AUC scores of 0.613, 0.744, 0.784, and 0.966. On newer datasets, its performance reduced to AUC scores of 0.533, 0.575, and 0.640.

1.2.2 DSP-FWA

The DSP-FWA method [6] originates from a research paper published in 2018 and then updated in 2019. It presents the theory of deepfake technology leaving distinguishable face warping artifacts (i.e. resolution inconsistencies between the face area and its surrounding context) in the resulting fake videos. In its research, the proposed model seeks to detect these inconsistencies to expose deepfakes by extracting the face region from each video frame and compare that area to its surroundings. Any areas that are caught by the algorithm as *regions of interest* are cropped to be used as input to a ResNet50-based SPPNet model.

The DSP-FWA research suggests using a CNN model from the ResNet architecture [23] with 50 layers. The idea is that the residual connections from the architecture will make the learning process effective, yet, 50 layers is enough depth as the classification-relevant information diminishes the more network depth increases. Apart from this, the suggested solution also uses a technique referred to as SPPNet [36]. SPP stands for *spatial pyramid*

pooling and is a technique used with CNNs. Traditionally, convolutional layers are followed by a pooling layer using one of several pooling functions, as described in [section 1.1.4](#), but instead, SPP suggests having multiple pooling layers. SPPNet came to solve the limitation that existing CNNs required images to be resized before used as input. He et al. [36] speculated that this constraint may reduce the recognition accuracy for images of arbitrary sizes and came up with the strategy of SPP to eliminate this requirement, enabling any image size to be used. The DSP-FWA model will output a prediction based on an averaged scores from the top third of video frames input.

In its original research, the DSP-FWA model was evaluated on ROC/AUC performance during three experiments where it received AUC scores of 0.932, 0.974, and 0.999 [6]. In the research from Li et al. [14], this model was tested on both older and newer deepfakes datasets and evaluated on ROC/AUC performance. On older datasets, it achieved the top AUC scores of all tested models with the values 0.930, 0.977, 0.997, and 0.999. On newer datasets, its performance reduced to AUC scores of 0.646, 0.755, and 0.811.

1.2.3 Ictu Oculi

The Ictu Oculi method [7] originates from a research paper published in 2018, suggesting that the physical signal of eye blinking is poorly reproduced in fake videos. In real videos, we expect to find spontaneous human eye blinking, and hence, the lack of such can be used to detect deepfakes. The Ictu Oculi model combines a CNN and a Long-Term Recurrent CNN (LRCN) to distinguish the eye state in a video and detect blinking patterns. Out of the single models selected for this experiment, the Ictu Oculi model is the oldest one. It was published about two years before this study was conducted.

The proposed CNN is a VGG16 model based on VGGNet architecture [22] with 16 layers for simplicity. The CNN model is used to locate and extract face areas in each video frame, converting these areas into discriminative features. The problem with using only a CNN is that the model wouldn't be able to analyze the regularity of the eye blinking. Instead, an LRCN is also leveraged because this sort of ANN is structured to increase the memory capacity of the model, making it possible to control when and how to update a state and in that way to incorporate relationships between consecutive video frames [1]. In this research, the state starts with the opening of the eyes and stops when they're closing. The LRCN can recall these gestures and detect the number of blinks per 60 seconds. The authors discovered that all deepfake videos tested in their experiment performed below their standard of the average blinking rate for a human being.

In its original research [7], the Ictu Oculi model was evaluated on

ROC/AUC performance during two experiments, using only CNN for the first experiment and using the CNN and LRCN together for the second experiment. When the model used CNN model VGG16 on its own, it received an AUC score of 0.98 that was increased to 0.99 when the LRCN was used together with the CNN. I have not encountered any research where this model is tested on more modern deepfakes datasets.

1.2.4 XceptionNet

The XceptionNet method [8] originates from a research paper published at the beginning of 2019 and then updated in August that same year. The XceptionNet model is a traditional CNN model constructed from the CNN architecture Xception [24]. This architecture was chosen by the authors because they desired a detection model that is able to achieve compelling results on images with weak to no compression, while still maintaining reasonable performance on low-quality images. In theory, this would mean that the model is more fit for handling videos of various conditions which is often the case in real-life situations.

The study conducts several experiments using video inputs of raw, high, and low quality to test the model's detection capability for various levels of image quality. In one variant of the experiment, the authors use full video frames as input and in the other variant, the inputs are preprocessed into crops around the center of the face, enlarged by a factor of 1.3. The cropped inputs turned out to be significantly easier to classify for the model. Additionally, the XceptionNet model demonstrated a greater ability to detect deepfakes from raw and high-quality images than from low-quality ones.

In its original research, the XceptionNet model was evaluated on binary classification accuracy during three experiments where it received 81.00%, 95.73%, and 99.26% [8]. In the research from Li et al. [14], this model was tested on both older and newer deepfakes datasets and evaluated on ROC/AUC performance. On older datasets, it achieved AUC scores of 0.540, 0.567, 0.804, and 0.997. On newer datasets, its performance reduced to AUC scores of 0.482, 0.499, and 0.539, which we can see is indeed below or just above random guess classification in the binary case.

1.3 Problem formulation

The aim of this project is to investigate if an ensemble of CNNs for deepfakes detection improves detection performance in comparison to single models. Ensembles have produced significantly better performance than single models when dealing with multiple complex problems [25] but this problem has yet to be explored in-depth for the classification problem of deepfakes detection.

When considering ensemble learning approaches, two relevant questions

arise. The first deals with the problem of what combination of accurate and diverse base-learners to use and the second concerns how to combine the base-learners outputs to achieve the best performing ensemble.

In this study, I will answer these questions by selecting multiple single CNN models proposed for deepfakes detection by previous research work. These single models will be implemented according to specifications provided in their corresponding research papers. Next, the single models will be trained on modern deepfakes datasets. Then, the single models will be evaluated on a test dataset containing new deepfake videos and their predictions will be combined into ensembles using both hard and soft voting approaches. Finally, ensemble performance is to be evaluated and compared to the outcomes achieved by the single models.

To summarise, the main goals of this work are to:

- Evaluate the detection performance of single CNNs.
- Evaluate the detection performance of neural network ensembles that combine those single models.
- Compare results from the ensembles to single-model achievements.
- Suggest the most promising solution for deepfakes detection.

1.4 Motivation

Fikse [12] gives an insight into the complexity of the deepfake phenomenon by suggesting the following categories of deepfake videos:

- **Technology demonstration deepfakes:** used to illustrate how deepfakes technology works and what it can do.
- **Satirical deepfakes:** humorous or mocking videos working as a form of political or social commentary. They're not made to be deceptive.
- **Meme deepfakes:** videos where faces are replaced as a funny idea.
- **Pornographic deepfakes:** exist in large numbers on the web and has the potential of being used as revenge pornography. Much of the early debate surrounding this deepfake technology focused on the lack of consent from people swapped into pornographic deepfakes.
- **Deceptive deepfakes:** videos made to deceive the viewer into believing a forged situation involving an actor of importance to the viewer. These videos would have political, legal, or other social effects, creating an illusion of some sort of video evidence.

Commentators have warned against the misuse of several of these categories of deepfake videos and how they could have serious political, social, financial, and legal consequences if used on social media for propaganda purposes, for the spreading fake news, or for attacking the reputation of public figures [37]. The videos could be tailored to reinforce false beliefs, stir up fear and hatred without factual grounds, or even reduce trust in video proof. It is not difficult to see how these warnings could come true when it

has already been mentioned how deepfake technology is evolving rapidly, making deepfakes more difficult to detect, especially for humans.

In 2015, Victor Schetinger et al. analyzed human ability to detect forged digital images, and their results indicate that people show insufficient skills at differentiating between altered and non-altered images [38]. The authors suggest that humans can easily be fooled by fake images as their participants only identified forged images about 46.5% of the time. Not only that; participants even frequently doubted the authenticity of real pictures. Another study from last year examined the performance of participants' distinction between real and fake videos [37]. Deepfake videos were among the fake videos tested and the research findings revealed that humans had a difficult time detecting when a video was fake. The authors likewise mentioned that in a real-life scenario, the audience would probably not actively be judging videos they see every day as being real or fake, especially if they already trust the publisher (or sharer) of the video, making it even more unlikely that they will recognize a deepfake.

If this experiment is successful it would contribute to further advancement in the field of deepfakes detection and the research and developer communities could continue to extend and improve from its conclusions.

1.5 Objectives

In this research, I study the problem of deepfakes detection using state-of-the-art ML solutions, and within this context, I investigate the possibility of using a neural network ensemble of CNNs to enhance detection performance. The primary aim is to compare ensemble performance with single model achievements. The secondary aim is to observe if I can find any indications of a relationship or link between detection performance and ensemble approach or base-learner diversity. In order to achieve these intentions, this work is pursued with the following objectives:

O1	Survey deepfakes detection methods and select a few suitable to use for this experiment.
O2	Do a review of ensemble learning approaches and decide on what alternative to use for this experiment.
O3	Run an experiment where several approaches to combine single models into ensembles are assessed. Investigate how ensemble performance might be affected by the collection of base-learners used and how their predictions are combined.

O4	Evaluate single model performances and compare them to ensemble performances for deepfakes detection. Select the best performing as the proposed solution.
-----------	--

I expect that ensembles will outperform single models on deepfakes detection performance, resulting in the final proposed solution to be an ensemble. Furthermore, I expect that the single CNN models will achieve a worse detection performance than what they manifested in their original research because of the progression of deepfake technology, just as in [14].

Inevitably, because of the limitations on this project scope, the results will likely leave room for both improvement and further research.

1.6 Scope

The purpose of this study is to compare deepfakes detection performance of ensembles to their base-learners, using modern datasets of deepfake videos. To keep this focus within the project time frame, some limitations on the scope are necessary:

- It's impossible to consider all existing deepfakes detection ML models for this experiment. Instead, only CNN models whose source code is publicly available will be considered.
- The source code must be written in the Python programming language and utilizing the same open-source libraries and frameworks so that the implementation of the single models is as straightforward as possible.
- To further simplify implementation, I will use the same configuration settings for the single models as was chosen by their original authors.
- The term *deepfakes* can be used to describe the specific deepfake technology and the videos it produces, or the term can be used more broadly to refer to any AI-generated impersonating videos [2]. In this study, I choose to use the specific version of the term which means that I will not include experimentation or evaluation on any fake videos except such created by the deepfake technology.
- This research aims at investigating how ensemble learning can be used to improve deepfakes detection performance. Any integrations with actual applications or systems are not part of this study.

Due to these stated limitations of the project scope, it can be expected that the results won't be fully demonstrative of how ensembles perform in relation to single models on the problem of deepfakes detection.

1.7 Target group

This study might be of interest to the following audiences.

1.7.1 Deepfakes detection community

There is a community of researches with the interest of improving deepfakes detection methods, this can be seen online in relevant forums as well as in public global challenges such as DeepFake Detection Challenge⁴ and Media Forensics Challenge⁵. As there is yet no silver bullet to detect deepfakes, any contribution to this research field will help that community to continue developing and evolving innovative technology solutions.

1.7.2 Programmers

Programmers interested in developing applications for alerting users when videos might have been subjected to deepfake technology forgery could benefit from this study by using its final proposed solution or simply any part of its findings. Examples of such applications are browser extensions and mobile apps.

1.7.3 Social networking companies

Deepfakes are of great concern to social networking companies as these types of content may be used maliciously as a source of misinformation or harassment when being spread across social media networking platforms. Deepfakes could both impact how users think about the legitimacy of information presented online and potentially affect the reputation of a social networking service company. As the companies behind these platforms plan how to deal with deepfakes, any research and suggested solutions that support the development of deepfakes detection might be of interest to these businesses and their continued advancement of detection systems.

1.8 Outline

The rest of this report is organized as follows: In [Section 2](#), the experimental setup and datasets are presented before I discuss reliability, validity, and ethical considerations. In [Section 3](#), the environment setup used for the experiment is described together with implementation details about training and evaluating the single models and the ensembles. In [Section 4](#), results are presented. In [Section 5](#), the results are analyzed and compared to the results and conclusions from prior work in the field. In [Section 6](#), I connect the results to the goals of this project and discuss the reasons why the results turned out as they did. Finally, in [Section 7](#), I present my conclusions together with some suggestions for future work.

⁴ <https://deepfakedetectionchallenge.ai/>

⁵ <https://www.nist.gov/itl/iad/mig/media-forensics-challenge>

2 Method

In this section, an overview of the experimental setup and the data used will be presented, followed by two subchapters where reliability and validity, as well as ethical considerations, are addressed.

The aim of this study is to investigate if ensembles composed of CNNs is a superior approach to single models for deepfakes detection. The answer to this question can only be found through the result of experimental trials, and consequently, in order to examine this, a controlled experiment will be conducted. A controlled experiment is defined by Langley et al. [39] as systematically varying some independent variables to test their impact on some dependent variables. Currently, the problem of deepfakes detection is commonly formulated, solved, and evaluated as a binary classification problem, which means determining the likelihood of a video to be real or fake [2], and such an analysis is straightforward to set up in a controlled experiment.

In this study, the experiment will be methodically managed in a controlled programming environment where the dependent variables are the evaluation metrics to measure, i.e. the results, and the independent variables are the training setting options, i.e. which model that is currently used and its configurations.

2.1 Datasets and data preprocessing

Generally, deep learning models, such as CNNs, are dependent on qualitative data to learn from and fine-tune its algorithm [1], which makes the availability of large-scale and modern deepfakes datasets a crucial factor in the development of deepfakes detection methods. Additionally, I find it necessary that the visual quality of the deepfake videos matches actual deepfakes circulating on the internet to assure results are as realistic as possible. Deepfakes with low visual quality are unlikely to be convincing in real-life scenarios and correspondingly, high detection performance on such videos may not be of strong importance. Accordingly, I have selected three modern, public deepfakes datasets that I will join into one comprehensive dataset for the experiment. This will ensure both the quantity of data required and the visual quality of the videos. Further, the full dataset (comprising of all three individual datasets) will be divided into train, validation, and test subsets with a ratio of 8:1:1, meaning that 80% of the videos are used for learning during the training phase, 10% are used for improving and tuning (validating) the model during the training phase, and the last 10% is used only for testing, also referred to as the *holdout set*. The holdout set contains videos the model has never seen before and provides the final evaluation of a model's performance.

Table 2.1 outlines an overview with basic information about the three individual datasets. The rest of this section will shortly describe each dataset and conclude by mentioning any data processing needed for the experiment.

Celeb-DF [40] is a collection of real and deepfake celebrity videos. The videos have been chosen to exhibit large variations in aspects of lighting conditions, video background, and diverse distribution of the characters' genders, ages, and ethnic groups. I use v2 of this dataset in the experiment, published at the beginning of 2020.

DeepFakeDetection [41] is provided by Google and Jigsaw for deepfakes detection research. They have used publicly available deepfake generation methods to create over 3000 manipulated videos from 28 actors in various scenes. The set also includes some original videos.

Deepfake Detection Challenge Training Set [42] is part of the Deepfake Detection Challenge. This challenge provides two public datasets: the full training set and a small sample training set. Both of them focus on providing videos with visual variability through diversity in areas such as gender, skin tone, age, head poses, and video background. The full dataset consists of a large collection of real and deepfake samples with just over 470 GB of videos. Due to both the time constraints of this study and limited computational resources, I will utilize the small sample training set in this experiment instead of the full set. The small sample training set consists of 400 videos: 323 deepfakes and 77 real ones. Even though this is a smaller set of videos, I consider it to be enough for the experiment when it's joined together with the other two datasets.

Dataset	Published	# Real	# Deepfakes
Celeb-DF (v2) [40]	2020	590	5640
DeepFakeDetection [41]	2019	363	3068
Deepfake Detection Challenge Small Training Set [42]	2019	77	323

Table 2.1: Basic information about the datasets.

Data processing is described as the process of manipulating samples of data to produce meaningful information to be used for an experiment [1]. In this study, the single models, along with the ensembles, are constructed of CNNs and these types of ANNs have an automatic feature extraction capability that enables them to learn representations from the data without the need for a separate processor [9]. Due to this nature of CNNs, the preprocessing of data can presumably be kept to a minimum [1] and hence, this phase of my

experiment will merely be composed of separating each video input into a sequence of video frames where, for each frame, the face area is detected and cropped into the actual input for the model. I will only use the face area and not the full-frame because it has been repeatedly shown that errors in fake videos are mostly located around the face area [10], and additionally, the authors of XceptionNet [8] recommend tracking face information instead of using full-frames to increase detection accuracy. There is no data augmentation used in this experiment. As for the single models' original research, the data preprocessing phases differed and I will, therefore, not use them in this experiment. Instead, the preprocessing phase will be as described in this section and it will be the same for all models.

The final process is the postprocessing of the data. Deepfakes detection solutions are generally solved at frame-level [2], which means the model determines the likelihood of individual video frames to be real or fake. That requires an extra step to combine the scores of all individual frames from a particular video into a video-level classification. This can be done by averaging the scores of all frames for a video and that average score will be the final output representing the model's prediction.

2.2 Experimental setup

This section will describe the experimental setup and its various stages.

2.2.1 Training setup

The first stage involves training the single CNN models according to the specifications and training settings stated in their research papers. In the original papers, the authors of each model have provided access to already trained models for those who wish to use these trained models directly for testing, eliminating the need to train the models from scratch. I will not directly use these provided trained models for evaluation and ensemble building as I consider it favorable for the results from this experiment to train the single models on newer generations of deepfake videos before evaluating their performances. This is because models solely trained on older deepfakes datasets might not perform well when detecting fake videos of better quality, as shown in [14]. However, I will utilize the pre-trained models and re-train them by exploiting the concept of *transfer learning*. Transfer learning is a common practice in the fields of computer vision and image processing where, instead of starting the learning process from scratch, a model already trained on some data is trained some more on different data to solve the same problem [11]. When using this practice, a model starts from patterns it has learned before and leverages that previous experience of detected features during the new training phase. It is shown that when applying the practice of transfer learning in an ML experiment, the detection of low- and high-level

features benefit, and this practice also helps to lessen the need for large amounts of training data [9].

For the training settings and configurations of the single models, they will have separate values for the number of training epochs, batch sizes, choice of optimization function, as well as the initial learning rate and learning rate decay, such as in their original research experiments. The Capsule model [5], built on the VGGNet CNN architecture with 19 layers, uses the Adam optimization function with a learning rate of 0.0005 and a learning rate decay that starts at 0.9 and increases slightly during training to a maximum value of 0.999. The Capsule model is trained on a mini-batch size of 64 data samples and runs for 25 training epochs. The DSP-FWA model [6], built on the ResNet50 CNN architecture with 50 layers, uses the SGD optimization function with a learning rate of 0.0001 and a learning rate decay of 0.9 that decreases if the loss value has stopped improving. The DSP-FWA model is trained on a mini-batch size of 56 data samples and runs for 20 epochs. The Ictu Oculi model [7], built on the VGGNet CNN architecture with 16 layers, uses the SGD optimization function with a learning rate of 0.01 and a learning rate decay of 0.9. The Ictu Oculi model is trained on a mini-batch size of 40 data samples and runs for 100 epochs. Finally, the XceptionNet model [8], built on the Xception CNN architecture with 71 layers, uses the Adam optimization function with a learning rate of 0.00001 and a learning rate decay that starts at 0.5 and increases during training to a maximum value of 0.999. The XceptionNet model is trained on a mini-batch size of 40 data samples and runs for 18 epochs.

In [Section 1.1.5](#), the topic of ensemble diversity and its importance was brought up. Achieving ensemble diversity can be done in three prevalent ways and in this study, two of them will be put into use. Presently, I have mentioned that the single models are built from different CNN architectures which makes them unique from each other and that they are using different configurations which adds to the diversity. As for the third way, it is to train the models on separate datasets and this will not be done in this experiment. That is because this ensemble experiment includes comparing base-learners' single performances and when that's the case, it's preferable that the single models have been trained on the same training set [28].

2.2.2 Ensemble learning setup

The second stage combines the now trained CNNs into stacked neural network ensembles using some different approaches. The most basic and convenient way to create the ensembles is to load the already pre-trained models after they have been saved during re-training in the first stage, and add them as ensemble members. At this point, I need to answer two questions: (a) which diverse base-learners to combine, and (b) how to

combine their votes (predictions) for each input to achieve the most accurate final output.

In order to answer the first question, I consider it to be beneficial to explore varying combinations of ensemble members and how these combinations will contribute to its performance. Evaluating different alternatives will allow for a prominent selection of ensemble members that will, in turn, enable the ensemble to be adequately compared against single model performances. Consequently, I will examine selecting all available base-learners and selecting only the best performing single models (based on the validation set) or only the single models with the smallest file sizes. I choose these three base-learner combinations because, as mentioned in [Section 1.1.5](#), the current practice is to combine all available single models, but Zhou et al. [28] suggest that this might not always lead to the most favorable outcome. Accordingly, I will compare this practice of combining all models to the practice of only combining some of the models, either because they are the best-achieving ones so I can examine if single model performance has a noticeable impact on ensemble performance, or because they are the ones with the smallest file sizes so I can examine if an ensemble requiring less computational resources still manages to accomplish a similar detection result.

As for the second question, I will study how much of an impact different prediction combination approaches have on the ensemble performance through using both hard (majority) voting and soft (weighted) voting. In hard voting, every member will vote for a classification (real or fake), and the class label receiving the most votes is chosen as the ensemble output. In soft voting, every member will again vote for a classification (real or fake) and their votes are combined after the ensemble calculates how much importance each of the model predictions will get.

2.2.3 Evaluation setup

The third and last stage consists of evaluating single model performance and ensemble performance. The ultimate goal behind this evaluation is to understand how well each detection solution would perform on unseen data, granting how adaptive the model is to real-life applications and ultimately, how reliable its predictions would be. The single models will be tested on the same test set as the ensembles to facilitate a valid comparison between ensembles and single models.

The evaluation metrics to be used when measuring model quality are primarily accuracy and ROC/AUC. The simple form of measuring accuracy on video-level will show how many videos the model predicted correctly. Here, the likelihood of a video being fake is simply computed as the average likelihood of its video frames. Besides that, the overall detection performance

will be evaluated by using the area under the ROC curve (AUC) score at the video frame-level. There is mainly one reason why I'm choosing to estimate AUC at frame-level: the single models used in this experiment were also evaluated on individual video frames in their original research, so the models already output a classification score for each frame. Therefore, using frame-level AUC will avoid any inconsistencies caused by using other approaches. Furthermore, this approach is also used by Li et al. [14] which will facilitate a compatible comparison between their results and mine.

Lastly, to fully evaluate the effectiveness of a model, I will inspect the associated confusion matrices and examine model values for sensitivity (true positive rate) and specificity (true negative rate).

2.3 Reliability and Validity

In this experiment, steps were taken to ensure results are reproducible (reliable) by making sure equal or comparable results can be achieved using the same methods under equivalent circumstances. The computational capacity needed to replicate this experiment will depend on software and hardware arrangements including various factors that can differ between systems. Utilizing GPUs is the best choice to make computation efficient and effective, however, running this experiment is not limited to solely computing on GPUs and a solution has been adapted to make sure computational work can be distributed among CPUs or GPUs with the same functionality.

In the experimental setup, all single models were trained, validated, and tested on the same subsets of data. They were evaluated under the same conditions and on the same evaluation metrics. Additionally, the ensembles were created with those same versions of the single models, and they were also evaluated on the same metrics after being tested on the same test subset. The implementation of this experiment has been documented and the source code is provided publicly. Reliability issues will likely originate from changes in this experimental setup, for example, selecting different single models or a different ensemble learning approach, working with other or additional datasets, changing any training settings and model configurations, or use other evaluation metrics. Furthermore, there is the possibility that the datasets used in this experiment might change over time and such circumstances could have an impact on detection performance results. Finally, it needs to be mentioned that this experiment was only conducted once and this means that there is a lack of confidence when it comes to potential variances of the expected result.

Measures have also been taken to ensure results are accurate (valid). The validity of this experiment relies heavily on the acquisition of data: how and where the data was acquired, how accurately it was labeled, how outdated the

data is, and what preprocessing it has been through before used as the model input. In this experiment, modern datasets were acquired and labeled from trustworthy sources to ensure validity. Then, the models were trained, validated, and tested on sequential frames extracted from single videos and each frame went through a simple preprocessing phase. It is possible that this preprocessing phase was too simplistic and that results could be improved if more advanced preprocessing was used, comparable to the single models' original research. Furthermore, the acquired datasets are imbalanced and contain much more deepfake videos than real ones, but techniques to prevent resulting distortions have not been applied. This might cause the evaluation metric of classification accuracy to become an unreliable measure of model performance [29], so the results from the confusion matrix and ROC/AUC metrics are considered more accurate. To prevent data leak, the videos were divided into subsets for training, validation, and testing before the videos were separated into frames. This is to prevent frames from the same video to get into several subsets which would create a scenario where the model is validated or tested on already seen data (because the frames come from the same video), making the results misleading. Despite this, the datasets still contain the same individuals in several of the videos which makes me wonder if it is possible for the ML models to get accustomed to the faces of these individuals, contributing to misleading detection performance results. Lastly, it needs to be mentioned that this experiment has only been tested in a controlled setting and was not followed by a field experiment in a real-life situation. Deepfake videos circulating on the web might be subject to other types of fabrications or manipulations that these models were not trained on, such as social media laundering, and anti-forensics techniques, which affect how the results would transfer to a real-life environment.

2.4 Ethical Considerations

The deepfake technology can have a huge ethical impact if it would be used for deceptive purposes such as some of the categories mentioned in [Section 1.4](#). In addition to those use-cases, there is an anticipation that deepfakes will become a way to attack ordinary people from the general public where a fake video would spread online to embarrass or offend the person fabricated into the video. If such a scenario would happen, both copyright infringement and the General Data Protection Regulation (GDPR) may assist a victim of a deepfake. Copyright infringement would protect the victim if the original media content used to swap the victim's face into the deepfake was protected by copyright law and then used without permission [43]. Additionally, a deepfake could count as personal data under GDPR since it, although fictional, still relates to an identifiable person and, therefore, that subject would have the right to request the creator or publisher of the deepfake

material to delete it, supported by article 17, “Right to erasure (‘right to be forgotten’)” [44].

Even though this work is supposed to contribute to the detection of deepfakes, I can’t dismiss the possibility of it being used to continue improving the original deepfake technology; it is nearly expected that any new public technological detection method could be used to the advantage of deepfakes developers. And, even though the original technology has a manifesto prohibiting unethical usage [45], the risk of that technology later being used improperly can’t be eliminated.

In this research, publicly available datasets were used for training, validation, and testing, so I do not need to pay attention to the privacy of the characters participating in the videos as I have agreed to the *terms of use* before downloading the datasets. However, if the results from this study would later be implemented in a real-life application deployed to check videos circulating on the web, considerations would need to be taken as to not violate the rights of the people participating in any gathered material.

3 Implementation

This section will describe the environmental setup used when conducting the controlled experiment presented in [Section 2.2](#). Further, it will define choices made during the various stages of the experiment, which were: (1) collecting and preprocessing of the data, (2) implementing and training the single CNN models, and (3) combining single models using different ensemble approaches. All source code used for the experiment is available at the associated GitHub repository⁶.

For the full experiment, each of the four single models was trained on two subsets of data: training and validation. Then, each of the single models was tested. Lastly, three different ensemble combinations using two separate voting strategies were evaluated, resulting in six ensembles. Therefore, the full experiment comprises training 4 (singles) models as well as testing 4 (singles) + 6 (ensembles) = 10 measurements of several evaluation metrics: accuracy, ROC/AUC, confusion matrix, sensitivity, and specificity. The experiment was not repeated because of time constraints.

3.1 Environmental setup

The first part of the environmental setup concerns the controlled programming environment. This experiment can run in any preferable Python 3 environment. In this study, the version of Python used was 3.7.7 and the experiment was conducted on a macOS operating system utilizing CPU for computation while working with Visual Studio Code as the code editor. With this setup, the full experiment process took about a week.

Python libraries and frameworks have been utilized in this experiment, a full list of required packages and their versions can be seen in *requirements.txt* on the repository. The packages of greatest importance were PyTorch⁷: an ML framework used to implement and train the single models, DeepStack⁸: developed for the purpose of ensemble building, and scikit-learn⁹: a machine learning library used in this experiment during evaluation. Python has several popular frameworks specialized in AI and ML. I chose PyTorch because it is recommended for research-oriented purposes as it provides an easy implementation of CNNs and supports changing the model's behavior dynamically (at runtime) to facilitate efficient model optimization. DeepStack is library-independent so it is straightforward to use with PyTorch.

The second part of the environmental setup concerns experiment

⁶ <https://github.com/angelicagardner/2dv50e>

⁷ <https://pytorch.org/>

⁸ <https://github.com/jcborges/DeepStack>

⁹ <https://scikit-learn.org/stable/>

management. In the context of ML, experiment management is the process of tracking metadata about each trial like code versions, training settings, model configurations, and evaluation metrics results. This metadata is collected for the purpose of sharing the results and reproducing the experiment. In this study, the Sacred¹⁰ framework was set up as a tool to provide basic infrastructure for the experiment management. I used Sacred to configure, organize, and log what happened during the different phases of the experiment, and it can also be used for reproduction.

The core class in Sacred is **Experiment**. It collects information about parameters and provides them with the expected main function when the experiment starts to run. The main function is assigned by decorating it with `@ex.automain`, provided that the Experiment-class has been instantiated as variable `ex`. Default configurations can be added as local variables using a defined config function decorated with `@ex.config`. If explicit configuration values are set for any execution of the experiment, they will be prioritized over the default ones without the need to change any defaults. Every time a trial is executed, information is collected about training settings, configuration values, any errors, and the results produced. This information is saved through a chosen observer, which could be a database or as in this study: basic file storage. Sacred creates the sub-directory of choice (e.g. `./results/experiments/`) to store the following files:

- `config.json` contains a JSON-serialised version of the configurations
- `run.json` stores the main information
- `cout.txt` holds the captured (terminal) output
- `metrics.json` consists of evaluation metric values

For easy replication of the experiment, I have included a script file (`run.sh`) that computes the complete experiment process from start to finish, the following phases are included: data preprocessing, training and testing the single models, combining and evaluating the ensembles. This script file can be executed in a Unix shell such as bash, but as Windows operating system doesn't have built-in utility to support `.sh` files, the following Python files need to execute sequentially to reproduce the same process:

1. `split.py`
2. `train.py`
3. `test.py`
4. `ensemble.py`

3.2 Collecting and preprocessing data

The data used during this experiment comes from the datasets presented in [section 2.1](#). To collect the videos, I followed instructions provided by the

¹⁰ <https://sacred.readthedocs.io/en/stable/>

authors of each dataset and accepted their *Terms of Use* before downloading the data.

Instead of using the datasets separately, I put all videos from each dataset into the same folder (`./data/videos/`) and created a CSV-file containing information about each video: the video id, which class it belongs to (0 for real and 1 for fake), and which dataset it originally comes from. This is to accommodate various ordering of the training data and to avoid any bias the CNN models might form when they're regularly exposed to the same persons and similar environments during training. The source code used in this step is included in the file `data_sorting.py`.

When the full experiment process runs, the file `split.py` is executed first. It starts with separating all videos into three subsets (train, validation, test), then it separates each video into a sequence of video frames: one frame from every 0.5 seconds in the video. From each frame, extracts the face area is extracted and saved as an image. In case more than one face is detected, both faces are saved as video frames. This is the complete data preprocessing phase and every time the file `split.py` is executed, it will create new subsets of the data but if it detects that videos have already been separated into frames, it will not repeat that part of the process.

For the remaining part of the full experiment process, a `CSVDataset` class from `dataset_loader.py` loads the video frames from the requested split and PyTorch's `DataLoader` class then provides an iterable over all files in that subset.

The folder structure and file organization related to this phase is seen in Figure 3.1.



Figure 3.1: Data preprocessing folder structure and file organization.

3.3 Implementing, training, and evaluating single CNNs

In this experiment, four single models built on different CNN architectures and with varying configurations were implemented from the source code and instructions obtained at each model's public GitHub repository, displayed in Table 1.2. In addition to this, the authors of each model have provided pre-trained models which I utilized by first instantiating a new class of that model (the class was taken from the source code), and then loading the pre-trained version before launching the training phase. The training was continued for the number of epochs specified for that model and stopped early if the model applied the technique of early stopping. Model configurations and training settings were mentioned in [Section 1.2](#) and [Section 2.2.1](#). Predominantly, I did not change any model configurations or training settings from those used in every model's original research, but one exception was the learning rate value for the Ictu Oculi model. In the original research, it was stated that the Ictu Oculi model is trained with a learning decay of 0.01 but when I used that value, after going through about half of the epochs, the loss had decreased so much it started to produce a NaN value which I presume relates to the vanishing gradient problem common for CNNs. When this problem occurs, the first solution to look for is reducing the learning rate which I did to first 0.001 and then 0.0001. A learning rate of 0.0001 turned out to be a suitable value because the problem ceased to exist after that. A second solution would have been gradient clipping, but I did not want to change the architecture of the model by adding a normalization layer, so I simply adjusted the learning rate setting. In addition to the single models' individual configurations and values for training settings, all models used the cross-entropy as loss function and softmax activation function in the output layer, replacing the CNN architectures standard fully-connected layers with binary output.

All single models were trained on the same training set and validated on the same validation set. During the training phase, training loss, training accuracy, validation loss, and validation accuracy were measured, and whenever a new best validation loss was achieved, the model was saved with its current weight values so, later, it is the best performing version of every model that will be used as ensemble members. The models were saved as .pth files which is a common PyTorch convention [46]. The model predictions on all video frames are also saved so I could review the predictions if I suspected something was wrong with the training settings, and also verify model diversity as I prepare for the ensembles. Model diversity was verified by me manually selecting 20 random video frames and controlling which scores and predictions the single models had produced for each frame. This was to make sure there is some diversity among the model predictions. The

training phase is found in the *train.py* file and for those who desire, this file can likewise be executed for only one selected model by running it once, or the script *train_all.sh* can be used on Unix systems to train all single models without the need to run the full experiment.

After the training phase was finished, the experiment process continued to evaluate the single models. This was done by feeding each model the video frames from the test dataset, and the model returned a related score y . The label 0 is associated with real videos and the label 1 with deepfakes, so if the score was lesser than or equal to 0.5 ($y \leq 0.5$), the frame was classified as belonging to a real video, but if the score was greater than 0.5 ($y > 0.5$), the frame was classified as belonging to a deepfake video. After the model went through all associated frames, it averaged its scores to output a classification prediction. This process is found in the file *test.py* and that file can either be executed for one selected model or the script *test_all.sh* can be used to test all single models on a Unix system without the need to run the full experiment.

3.4 Creating and evaluating ensembles

In the last phase of the full experiment process, the single models are combined into ensembles using six different strategies.

In the DeepStack library, there are two ensemble classes: StackEnsemble representing a stacked ensemble with hard (majority) voting, and DirichletEnsemble presenting a stacked ensemble with soft (weighted) voting. The DirichletEnsemble calculates weights for its ensemble members giving each a rank of how important its prediction is to the ensemble output based on how the member performed on the validation dataset. The weight optimization is performed with a randomized search based on the Dirichlet distribution. Both of these classes were used in the experiment to create the ensembles displayed in Table 3.1.

Ensemble class	# of members	Base-learners	Voting
StackEnsemble	2	Best-performing	Majority
DirichletEnsemble	2	Best-performing	Weighted
StackEnsemble	2	Smallest file size	Majority
DirichletEnsemble	2	Smallest file size	Weighted
StackEnsemble	4	All	Majority
DirichletEnsemble	4	All	Weighted

Table 3.1: Information about the six ensembles used in this experiment.

For every ensemble, the class is first instantiated and then the saved single models are loaded and added as members of the ensemble. Before adding a model as a base-learner, I call the PyTorch **model.eval()** function to change the model mode from training to evaluation because PyTorch ANN models are, by default, in training mode. This was also done before testing the single models. Failing to change the mode from training to evaluation might cause inconsistent results [46]. Additionally, when adding an ensemble member, the ensemble needs to know the model's predictions on the test set that the ensemble will be evaluated on, so those predictions are provided with the model. This can be done by either testing every single model on the test set before adding it as a member to get its predictions, or as was done in this experiment, test all single models on the test set first and save their predictions, then load those predictions before adding each model as an ensemble member. If the ensemble is of the class `DirichletEnsemble`, the ensemble also needs to be provided information about how accurate the model was at validation data so it can distribute its weights of importance accordingly. In this experiment, that was accomplished by loading the model's predictions on the validation set as well as the validation set labels, and providing these to the `DirichletEnsemble`.

After base-learners were added, the ensemble goes through all provided information to learn how to combine its members' votes before it makes its own predictions on the test set. When ensemble training has completed, information is returned about the ensemble detection performance along with the performances of its base-learners.

4 Results

The controlled experiment described in [Section 2](#) was designed to determine if using a neural network ensemble produces a better performance on deepfakes detection than single CNN models. In this section, the outcomes achieved from that experiment will be presented. The results from the single models are acknowledged first, followed by the results produced by the ensembles. Measuring single model performances will lay the foundation for answering the research question of this study as well as provide insights into what might make an ensemble perform better or worse than single models.

All results presented in this section are provided in a per-video fashion by averaging all video frame predictions for one video, except the evaluation metric of ROC/AUC which is given per-frame for reasons mentioned in [Section 2.2.3](#). The collected evaluation metrics reveal comparable properties such as how accurate a model is at detecting deepfake videos and how often it fails at this task, along with whether the model ever makes the mistake of incorrectly classifying actual real videos as fake.

4.1 Single model performances

To begin with, Table 4.1 contains file sizes for the trained single CNN models. The single CNN models were saved during the training phase when the model reached its best validation loss value. Model file size might not be of high importance for the main objectives of this study, however, I have added this into the results section to demonstrate any potential or weakness the single models have in terms of actually being deployed in real-life applications. An ensemble would need to deploy all of its members, which often turns out to be computationally expensive. Additionally, two ensembles were created by combining the single models with the smallest file sizes and in order to achieve this, file sizes are important to know.

Model	File size
Capsule	558,3 MB
DSP-FWA	94,3 MB
Ictu Oculi	531,7 MB
XceptionNet	83,5 MB

Table 4.1: File sizes for single CNN models and the ensemble.

In order to analyze and compare the single models, Table 4.2 shows single

model accuracy and loss for training and validation results during different epochs of the training phase. The number of total epochs differs for every model due to these values initially being selected by the original authors of each model [5], [6], [7], [8]. The epoch values range from between 18 to 500 epochs and I have used these same settings in the experiment when re-training the single CNNs. Consequently, because of this difference, I have only presented accuracy and loss values for the first, tenth, and last epochs. The values were saved at the end of an epoch.

Model	Epoch	Train acc.	Train loss	Val acc.	Val loss
Capsule	1	89.08%	14.73	82.14%	21.44
	10	95.02%	2.08	82.14%	23.43
	25 (Last)	94.92%	1.46	81.11%	3.13
DSP-FWA	1	73.29%	0.49	50.00%	1.00
	10	93.23%	0.18	50.03%	0.89
	20 (Last)	93.23%	0.18	72.31%	0.65
Ictu Oculi	1	91.01%	0.25	82.36%	0.52
	10	97.67%	0.05	81.84%	1.80
	100 (Last)	98.08%	0.05	86.38%	0.34
XceptionNet	1	89.00%	0.25	82.08%	0.47
	10	94.51%	0.04	56.83%	0.63
	18 (Last)	100%	0.00	45.84%	1.23

Table 4.2: Single models training and validation accuracy and loss results for different epochs during the training phase. The top results are shown in bold.

Then, the single models were evaluated on the test set containing previously unseen data samples. Their ROC curve achievements and calculated AUC, shown in Figure 4.1, will be used to analyze and compare the single models. The label on the bottom-right corner displays the model name and its AUC value. The figure also shows how the performance of a random classifier would look like. In Figure 4.1, the y-axis labeled “True Positive Rate” represents how well a model performed at correctly identifying deepfakes

while the x-axis labeled “False Positive Rate” represents to what extent the model incorrectly classified real videos as deepfakes.

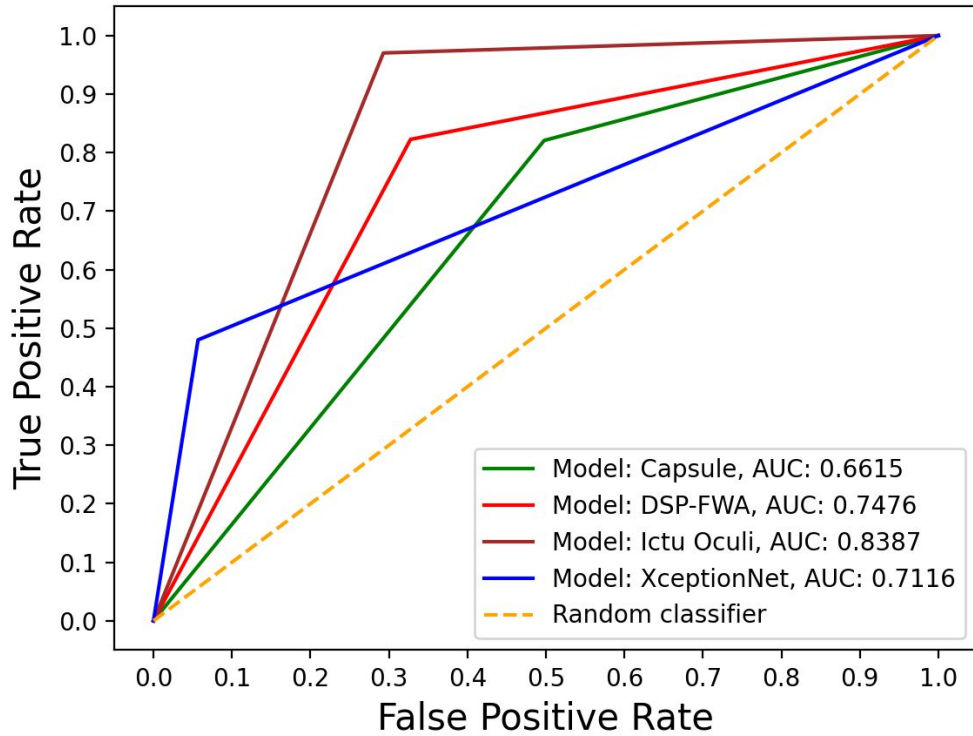


Figure 4.1: Comparison of ROC curve performances and AUC values for single models evaluated on the holdout set.

During single model evaluation on the test set, a confusion matrix estimated how the model classified each video to provide a detailed performance view. Figure 4.2 displays these confusion matrices, one per single model, and Figure 4.3 show a comparison of all matrices in one figure. The values for the confusion matrices were measured in a per-video fashion, meaning that for all video frames, the score was averaged and that average score became the final classification output for that video. The test set contained 1306 randomly selected videos from when the full dataset was split into three subsets (i.e. train, validation, and test). For every confusion matrix, the top-left box represents how many real videos the model correctly classified as real and the top-right box shows how many deepfake videos that were missed by the model and incorrectly classified as real. The bottom-left box displays how many real videos that the model incorrectly classified as deepfakes, and the bottom-right box shows how many deepfake videos that were found by the model and correctly classified as deepfakes.

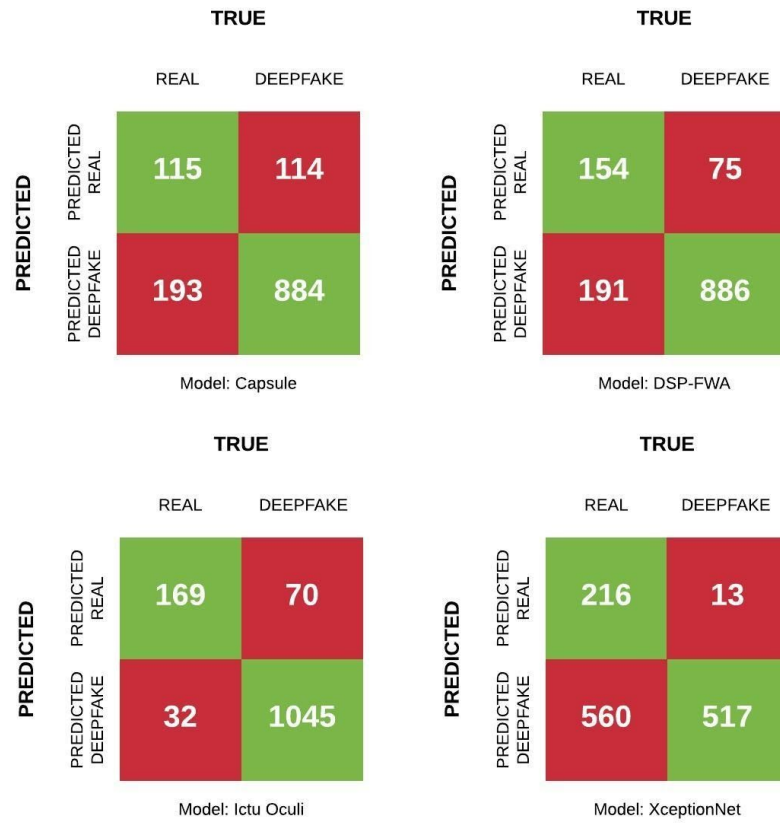


Figure 4.2: Confusion matrices showing video-level true and false classifications made by all single models.

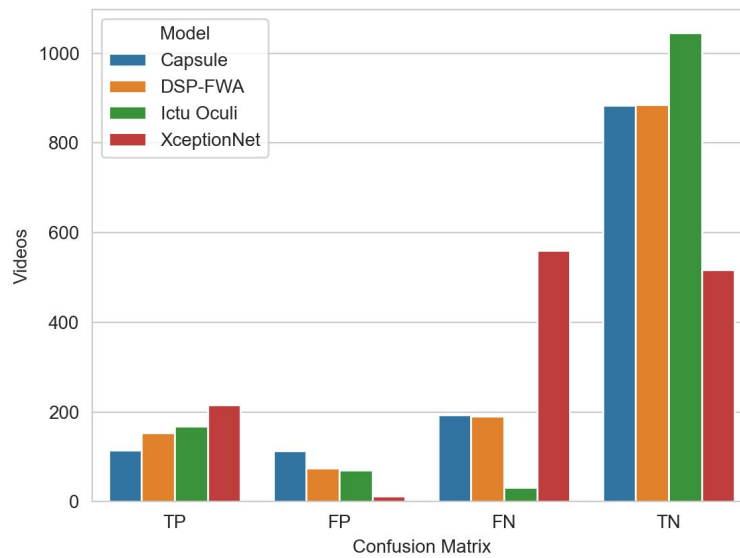


Figure 4.3: Comparison between the single models' confusion matrices.

Accuracy was estimated simply by calculating all correctly classified data samples, divided by all data samples (correct and incorrect). Furthermore, from the confusion matrices, we get information about the sensitivity and specificity of each model. Sensitivity represents the ability of the model to correctly detect deepfake videos (True Positive Rate), as also demonstrated by the ROC curve at the y-axis. A model with a sensitivity value of 1 could identify all deepfakes. Specificity represents the ability of a model to correctly classify real videos as real (True Negative Rate). A model with a specificity value of 1 could identify all real videos. In Table 4.3, accuracy, sensitivity, and specificity values are shown for all single models.

Model	Test acc. (%)	Sensitivity	Specificity
Capsule	76.49%	0.82	0.50
DSP-FWA	79.63%	0.82	0.67
Ictu Oculi	92.24%	0.97	0.70
XceptionNet	56.12%	0.48	0.29

Table 4.3: Binary detection accuracy (%) of the models as well as sensitivity and specificity. The top results are shown in bold.

Finally, as mentioned in [Section 1.1.5](#), ensemble diversity is crucial to create ensemble robustness and strong accuracy performance. Therefore, in order to get a slight understanding of whether these selected single models are independent enough to a certain extent that they can effectively be combined into an ensemble, I examine the single models' prediction scores for 20 randomly selected video frames from the test set. Figure 4.4 displays a visualization of the different scores for these video frames, showing a small image of the video frame together with its label (Real or Fake) and four points: one point for each model prediction. If the point has a blue color, it means the model classified that video frame as deepfake. On the contrary, if the point has a yellow color, it means the model classified that video frame as real. The numeric score on top of the point shows how certain the model was at its prediction. When the number is either 0 or 1, without decimals, the model was completely confident of its prediction. Accordingly, when the score starts to get closer to 0.5, it means the model was less confident about its prediction. If several models predict similar scores for a video frame, this can be seen by the four points being of the same color and having a similar numeric score. Correspondingly, any dissimilarities in predictions are displayed by different colorings and diverse numeric scorings.

This completed the presentation of evaluation metrics for single models.

Following, the single models were combined into ensembles.

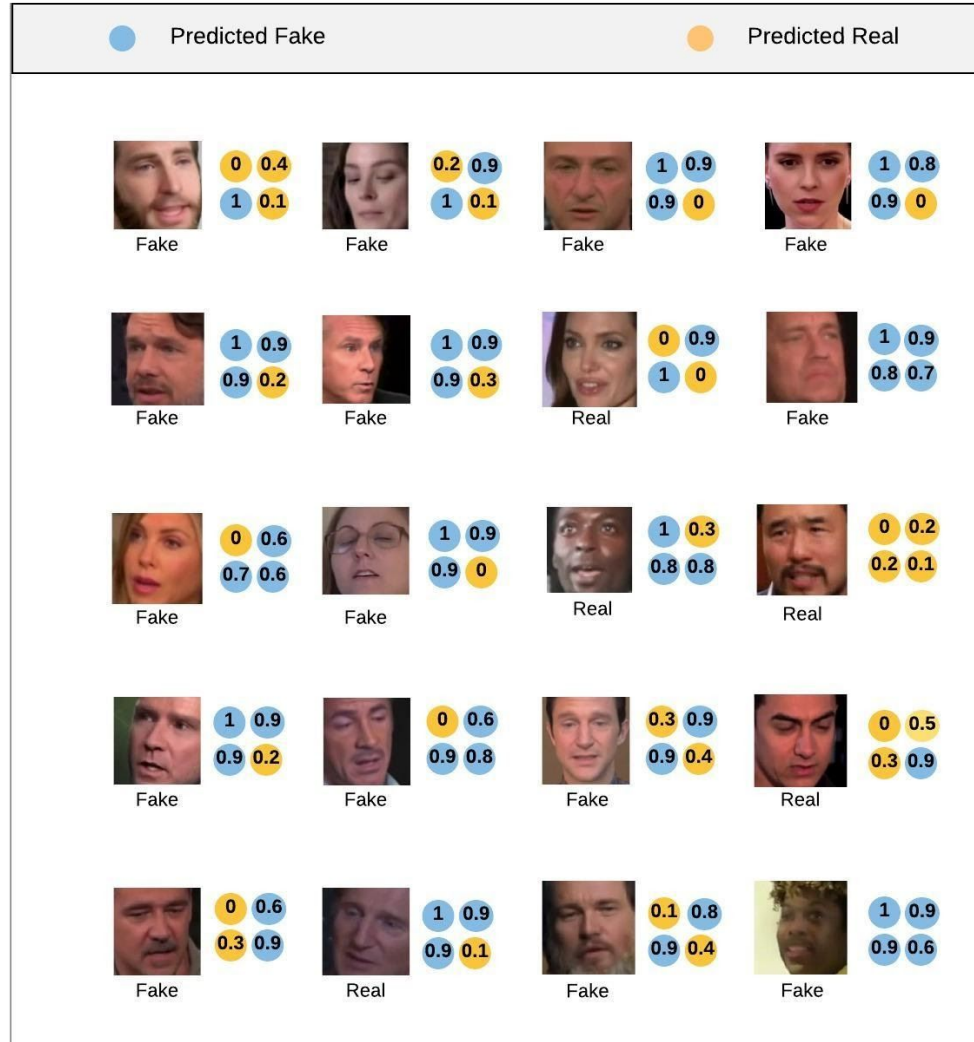


Figure 4.4: Visualisation demonstrating the prediction distribution between the single models for 20 video-frames.

4.2 Ensemble performance

During the experiment, six different approaches were attempted to combine the single CNN models into stacked ensembles. These approaches were mentioned in [Section 3.4](#).

In Figure 4.5, the AUC scores achieved when combining the two best-performing single models can be seen in comparison to its base-learners, utilizing hard versus soft voting.

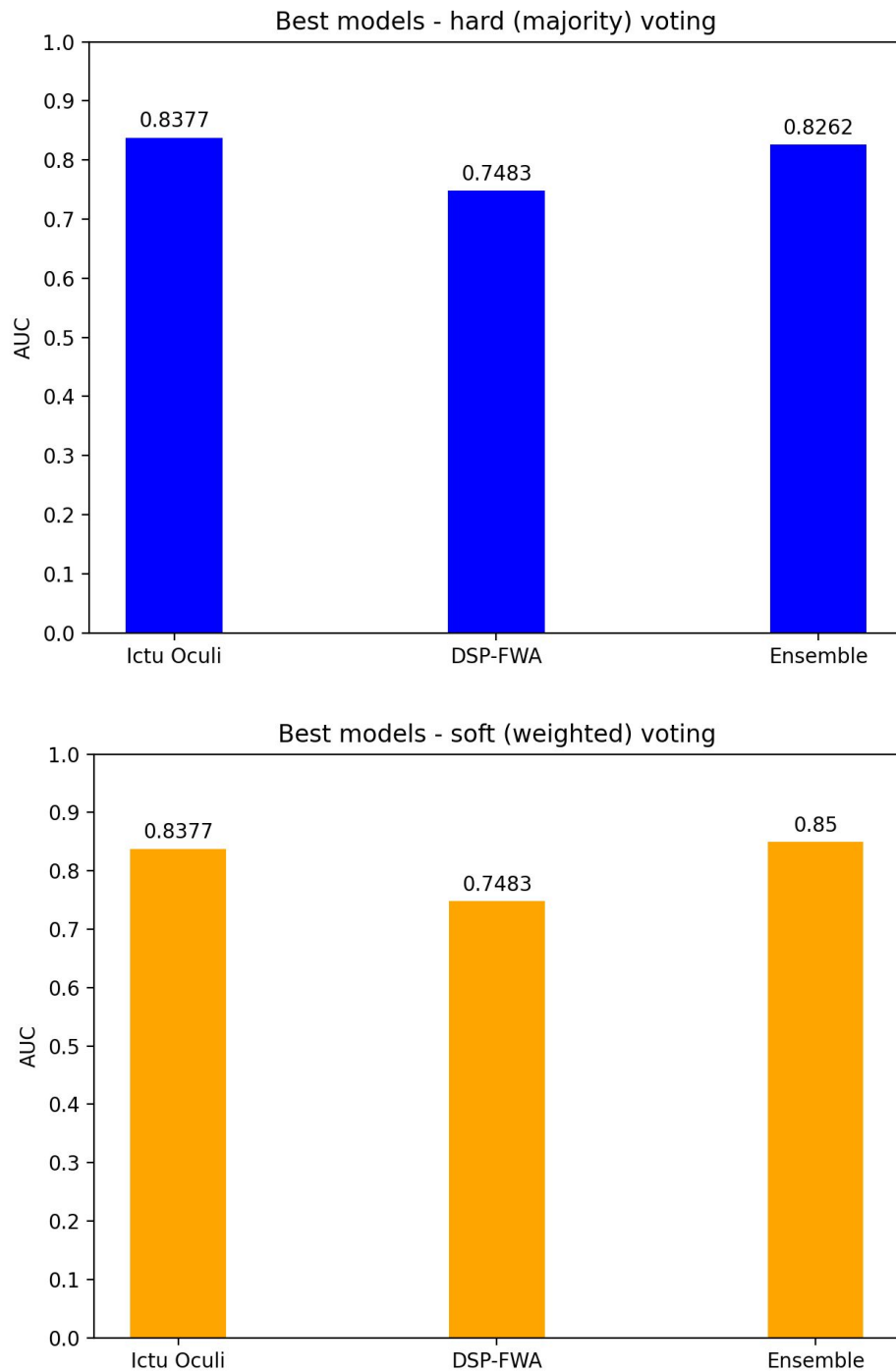


Figure 4.5: AUC scores for single models and ensemble when the ensemble approach was to combine the two best-performing single models.

Next, the AUC scores achieved when combining the two smallest sized single models can be seen in Figure 4.6, utilizing hard and then soft voting.

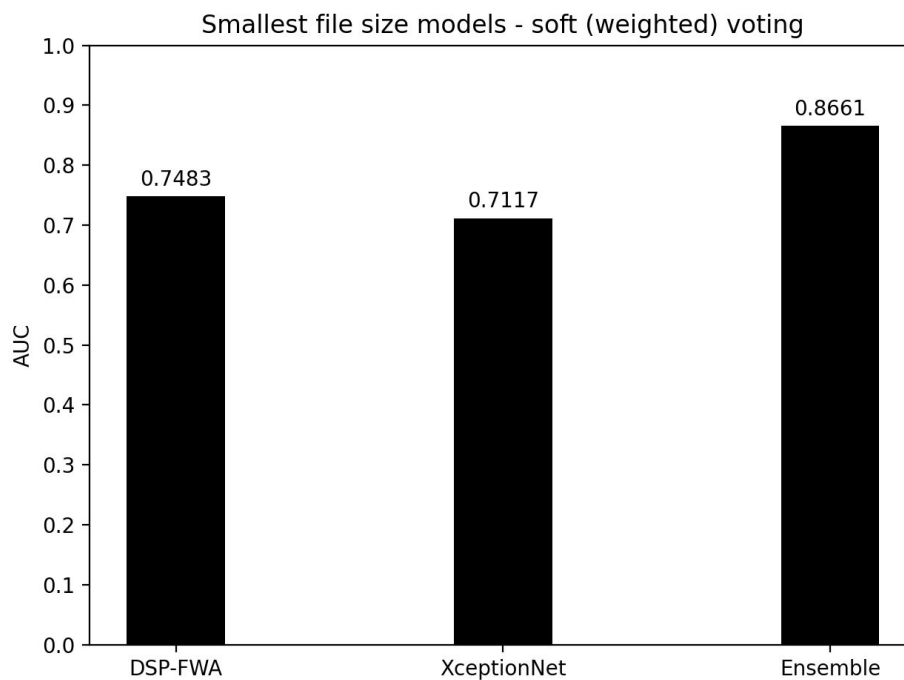
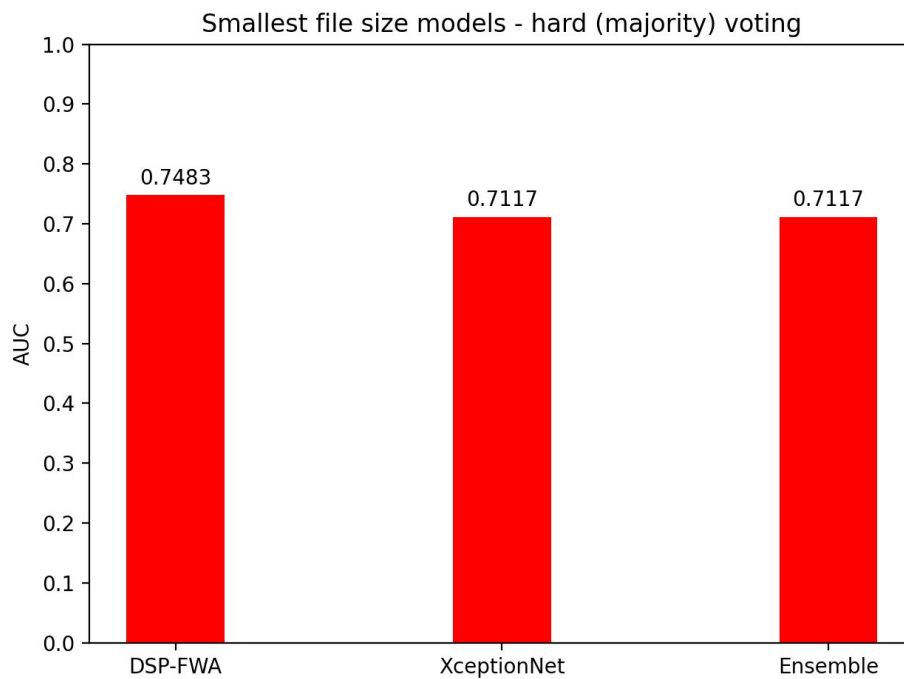


Figure 4.6: AUC scores for single models and ensemble when the ensemble approach was to combine the two single models with the smallest file size.

Lastly, the AUC scores achieved when combining all single models can be seen in Figure 4.7, utilizing hard and then soft voting.

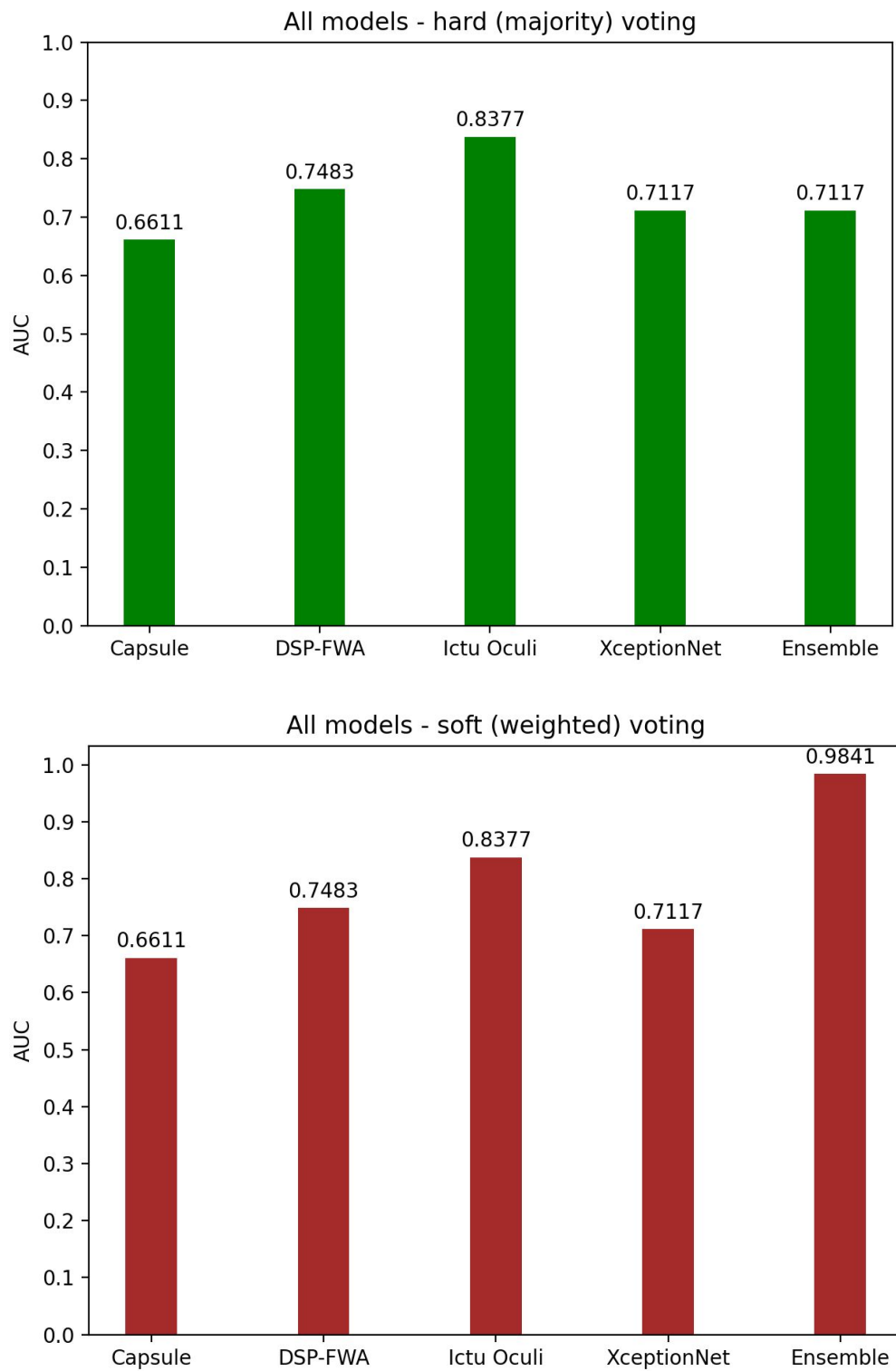


Figure 4.7: AUC scores for single models and ensemble when the approach was to combine all available single models.

Out of the ensemble approaches tested, the highest AUC score was achieved when ensembling all available single models and using soft (weighted) voting. Table 4.4 shows a summary of the AUC scores for all single models and the ensemble, together with how the ensemble distributed its weights of importance when combining the base-learner predictions. The larger the weight, the greater importance will the ensemble put on that model's classification of the input.

Base-learner	Weight by ensemble	AUC score
Capsule	0.3115	0.6611
DSP-FWA	0.3208	0.7483
Ictu Oculi	0.3159	0.8377
XceptionNet	0.0519	0.7117
Ensemble	-	0.9841

Table 4.4: Displaying the different weights of importance given to each base-learner from the ensemble when using soft (weighted) voting.

5 Analysis

In this section, the results presented in [Section 4](#) will be analyzed. First off, the analysis will concern reflections and opinions about single model performances, and secondly, compare these accomplishments to the different ensemble performances.

5.1 Single model performances

Firstly, in Table 4.1, the single model file sizes are displayed and it is clearly seen that models built on the VGGNet CNN architecture (Capsule and Ictu Oculi) have a larger file size than models based on the other CNN architectures. Training these models also took a noticeably longer time. This confirms what is mentioned in [Section 1.1.4](#) about the drawbacks of VGGNet which makes deployment in real-life applications impractical. Both Ictu Oculi and Capsule surpassed the other models in regards to detection accuracy on the validation set, but only Ictu Oculi stood out as the best-performing single model on the test set by achieving the highest test accuracy and AUC value. Considering this, it can be suggested that the performance of Ictu Oculi balances out the drawback of its large file size, but that's not the case for Capsule where a similar performance was reached by other models.

Training the single models with the concept of transfer learning was straightforward and from the metrics measured during this training and validation phase displayed in Table 4.2, I can notice some observations. First, the Capsule model has a much higher loss value than the other single models. This indicates that the Capsule model was very confident that it makes correct classifications, even when it makes incorrect ones. This made the loss value increase substantially. It seemed to struggle with this issue throughout the training phase and even though it managed to decrease its loss, this value was still much larger than the other single models. The random sample of video-frames displayed in Figure 4.4 seems to confirm this idea of the Capsule model being very confident as it continuously showed predictions of 0 or 1 without ambiguity. Secondly, the XceptionNet model started to overfit and memorize the training data at the end of the training phase, even though it only iterated for 18 epochs. This can be seen by the model achieving 100% detection accuracy and 0 in loss value on the training set, while validation loss increased and detection accuracy was low. This model would benefit from using the technique of early-stopping to prevent overfitting. In fact, the DSP-FWA model, which uses the technique of overfitting, actually stopped training after running through about $\frac{3}{4}$ of its epochs because it started to show signs of overfitting. This way, the issue did not seem to influence its detection performance but, unsurprisingly, the XceptionNet model achieved

the lowest detection accuracy on the test set: 56.12%, seen in Table 4.3, most probably because of the overfitting that led to bad generalization capability. Lastly, the model Ictu Oculi achieved the top validation accuracy of 86.38% and likewise achieved the top test accuracy of 92.24%, seen in Table 4.3, which is a valid sign of good generalization capability. This confirms the suggestion by Smith [18] that smaller batch sizes lead to an increase in generalization capability as Ictu Oculi used the smallest batch size. XceptionNet also used a small batch size, but as it overfits on the training data I am not considering its results in this context.

Among the evaluation metrics selected, the ROC curve performance and its accompanying AUC value is the best way to evaluate model performance on imbalanced datasets [29]. As seen in [Section 2.1](#), the deepfake datasets used in this experiment are clearly imbalanced as deepfakes represent a large amount of the videos. Figure 4.1 displays the ROC curve and AUC for all single models and I can conclude that all single models reached a detection performance above the diagonal line (0.5). If any of the models had fallen below that diagonal reference line, it would mean that model has no discriminability (i.e. the model doesn't possess the quality to distinguish between real and deepfake), and as a consequence, that model would not have been a good fit to use for the ensemble. The Ictu Oculi model with the largest AUC value of 0.8387 was best at distinguishing between real and deepfake videos, presenting a better average performance than the other single models. In Table 4.3, the sensitivity and specificity values are shown for all single models, and the fact that the Ictu Oculi model reached top results for these metrics support the opinion that this model has the best-distinguishing capability. The final consideration I can take from Figure 4.1 is that the XceptionNet model has a ROC curve progressing more to the left without reaching the top, indicating a high false-positive (FP) rate. Clearly, this model performed poorly when differentiating between the two categories and this can also be seen in the model's low sensitivity and specificity values in Table 4.3. The XceptionNet model especially seems to struggle to recognize real videos and instead classifies them as deepfakes.

The confusion matrices displayed in Figure 4.2 and Figure 4.3 provides an insight into how the single models have classified the inputs. Here I can see what is mentioned above about the XceptionNet model incorrectly classifying real videos as deepfakes. This model did not miss to detect as many deepfakes as the other models but instead seems to have classified the majority of the inputs as deepfakes. This issue could either have risen from the dataset being imbalanced or it could be an issue from overfitting where the model missed the true patterns and instead picked up random noise. The Capsule and DSP-FWA models also show that they classify many real videos as deepfake, but not nearly as many as XceptionNet. The Capsule and DSP-FWA models have also failed to detect several deepfakes, resulting in

the lower accuracy results revealed in Table 4.3. In that table, especially the Capsule model showed poor performance at distinguishing real videos, observed at the low specificity value. Last, the Ictu Oculi model presented the highest accuracy of 92.24% in Table 4.3 and also performed best on ROC/AUC in Figure 4.1. This model has a sensitivity value of 0.97 and a specificity value of 0.70 - in other words, it correctly identifies deepfakes 97% of the time but also incorrectly classifies 30% of real videos as fake. The sensitivity and specificity of the single models were examined. Except for the XceptionNet model, all single models exhibited near indistinguishable performance differences despite notable variability in evaluation metric values. I am disappointed over the fact that all single models received low specificity values indicating they perform badly at classifying real videos as real. I do not consider a high false rate to be appropriate for real-life applications. Other than this, the results did hardly reveal any clear trends related to the sensitivity and specificity of the CNN models.

There seems to be this common issue among the single models to classify real videos as deepfakes rather than just failing at detecting deepfakes, which makes me believe this issue presumably comes from the training phase and the data used to train the models. From the validation results in Table 4.2, it does seem like the models were learning the right patterns during training as validation accuracy steadily increased and validation loss decreased on the whole, with the exception of XceptionNet after it started to overfit. Additionally, the test results from this study seem to correspond with the results from Li et al. [14] more than the results from their original research studies, as seen in Table 5.1.

Model	Results from original research	Average results from [14]	Test results from this study
Capsule	94.4 - 97.6%	69.4%	76.4%
DSP-FWA	93.2 - 99.9%	87.4%	79.6%
Ictu Oculi	98.0 - 99.0%	-	92.2%
XceptionNet	81.0 - 99.2%	63.3%	56.1%

Table 5.1: Comparison of single model results from three studies.

The single models were never re-trained in the experiment from [14] as opposed to this study, and I did expect the models would improve their performance after being trained some more on newer deepfakes datasets. However, they still did not reach close to the performance of their original studies. This might be an indication that the single models are more

dependent on their individual data preprocessing phases than I initially thought, as I did not include those phases in the re-training but used a simple and universal preprocessing phase for all models. This is the only variation I can think of concerning the implementation of the single models as I used the same source code, model configurations, and training settings as the original research studies.

Finally, Figure 4.4 confirms my idea that the single models will provide enough diversity for it to be motivating to ensemble them. The scores for the random video frames displayed in this figure show that different models provided a slightly different score for each frame, seen by the fact that the numeric scores on the points are somewhat distributed, and at times, even the classification varied. If all numeric scores were perfectly joined, the idea to ensemble these models would not be reasonable because of the lack of diversity. Additionally, even though I consider it to be arguable how beneficial these single models would be on their own, in Figure 4.1 at least all of them showed a distinguishing capability above random classification.

5.2 Ensemble performance

For the ensemble performance, I was surprised to find that the majority of approaches hardly or just slightly outperformed the best-performing single models. For small ensembles, I found a slight advantage when using soft (weighted) voting as the approach for combining the predictions, but none of these approaches was competitive enough. For the larger ensembles that combined all of the models, the stacked ensemble even performed worse on ROC/AUC than the Ictu Oculi single model when using hard (majority) voting, as seen in Figure 4.7. However, when using soft (weighted) voting, the ensemble received a significantly improved result for deepfakes detection with an AUC value of 0.9841, which is substantially better than its base-learners, an increase of 15% from the best-performing single model. I interpret this result as that the majority of the single models were weak in their prediction accuracy, leading to the ensemble also making incorrect predictions when choosing the majority vote. Alternatively, when the ensemble went through the validation results of all its members and distributed its weights of importance thereafter, it could choose the prediction that came from the most important base-learners. The weight distribution from the ensemble, along with the AUC results is seen in Table 4.4. Interestingly, the weight distributions were quite equal between Capsule, DSP-FWA, and Ictu Oculi models, with the DSP-FWA model receiving the most weight of importance - and not Ictu Oculi which I would assume because of it being the best-performing single model on the validation set. This is one of the disadvantages of using ensemble learning, i.e. the reduction in interpretability which makes it more difficult, with the increased

complexity, to understand ensemble choices.

Assuredly, after analyzing these results, they clearly show that the approach of ensembling awards in terms of performance and that choosing the most appropriate strategy is crucial. It makes no sense to combine single models if there is no sufficient increase in performance on the testing data. When an appropriate ensemble approach is chosen, it will help both the accuracy of the deepfake detection and the quality of the predictions, estimated by AUC value. This result reinforces the importance of ensembles to obtain improved performance, but also the importance of choosing the right ensemble strategy.

6 Discussion

The primary aim of this study was to compare ensemble and single model performances for deepfakes detection. The secondary aim was to find indications of any connection between detection performance and the selected ensemble approach. These two questions were answered through the conducted experiment.

As for the primary purpose, this study did not prove that all ensembles automatically outperform single models, unlike previous work in related research fields [11], [32], and [34]. The results from [11] show that for small ensembles, it was useful to only select the best-performing base-learners, and in every case, ensembles outperformed single models. Also in [34], all ensembles outperformed single model performances and the final proposed solution was an ensemble combining all available single models. In contrast to these findings research, this experiment did not show these same conclusions. However, it needs to be considered that the results produced in this experiment have its limitations because of the fact that base-learners were only four single models, a very small amount. The ensemble in [32] surpassed the performances of all single models which reinforce the importance of ensembles, just as this study, but as they did not create several ensembles, it is difficult to know if they would have found occasions where this was not automatically the case like shown in this experiment.

On the other hand, in the research from He et al. [33], the best-performing single model exceeded two of the ensembles in classification performance, confirming the conclusions from this study. And additionally, just as in this study, their final best solution was an ensemble that generated a more accurate prediction than any of the single models used.

As for the secondary purpose, the results clearly showed a connection between detection performance and the selected ensemble approach where the combination of ensembling all available models and using soft (weighted) voting improved performance significantly.

In addition to these research goals, there are some considerations I want to highlight. Firstly, the training set used in this experiment consisted of approximately 8000 real and deepfake videos which I considered sufficient, but it is possible it was not. Additionally, I did not use data augmentation or other data preprocessing techniques to support the single models learning in this experiment. Inescapable, algorithmic behavior and performance are influenced by the experimental setup and choice of the training dataset. The fact that the dataset was imbalanced, meaning that the class distribution is not uniform among the classes, will contribute to the models being unevenly disposed to the two classes. Also, in real-life situations, this imbalanced condition will most probably be the opposite, i.e. the detection models will be exposed to more real videos than deepfakes. There is also the issue of videos

containing more than one person. Currently, the experiment captures all faces in a video frame to use as input for the model but there is an issue of when only one face has been tampered with in a video, but the other face(s) are real. As the model will output the average prediction of all video frames, this situation might lead to the model classifying a video as real even though it did contain a fake face in a small amount of the video frames. That situation has not been addressed in this study. The same holds true for when a detection model is looking for certain deepfakes features that seldom appear in a deepfake video. If that deepfakes feature is visible to the human eye, a human would look at the full video, and even though it might be of realistic visual quality, that person would still understand it is a deepfake video because of a few seconds of visual error. The question is if this would be the case for an ML detection model when averaging the frames scores. All of these three considerations might require the binary classification to be extended to multi-class classification as well as local detection to fully handle the complexities of real-life video forgeries.

Secondly, Ictu Oculi was the best-performing single model and it did neither use a more modern CNN architecture than the other models or very deviating model configurations and training settings. Yet, its research is based on the idea that the physical signal of eye blinking is poorly reproduced in deepfakes. Could the results from this study indicate that eye blinking is a good clue to look for when identifying deepfakes? It might be more challenging to eliminate this clue than to remove other defects created by deepfake technology, such as blurriness or discolorations.

To conclude this discussion, the results of this experiment appear to somewhat confirm the results from the research of Li et al. [14]. Three of the four selected single models were tested in their research and all of them performed distinctly worse at deepfakes detection than in their original studies, as mentioned in [Section 1.2](#), which was also seen in this study. That confirms their conclusion about the continued need for deepfake detection improvements as the difficulty level for detection is raised the more deepfake technology produces videos of higher quality. The results from this study certainly imply that deepfake technology is developing fast where one can't use a few years old detection models and expect modern-level performance.

7 Conclusion

Being able to detect whether a video contains manipulated content is nowadays critical given the significant impact of videos in everyday life and online communication. Consequently, this study focused on investigating the detection of face manipulations in video sequences, targeting fake videos generated by deepfake technology. While the majority of related work concerning deepfakes detection focus on highlighting the performance of a single novel approach or method, this work has focused on the comparison of performances of single models and ensembles.

The goal of this experiment was to combine multiple single CNN models into stacked neural network ensembles of different approaches and compare deepfakes detection performance between ensemble and single models. Based on this goal, a controlled experiment was conducted where single models and various ensembles were evaluated to make this comparison. Building on previous work in deepfakes detection, four single model methods were selected and implemented, re-trained through the concept of transfer learning on three modern deepfakes datasets, and finally combined into six different ensembles.

The results show a significantly different detection performance between what the single models demonstrated in their original research and what they achieved in this study. Furthermore, the ensembles showed notably different detection performances, some of them even worse than single model performance. Nevertheless, the final proposed solution was a stacked neural network ensemble combining all available single models and utilizing soft (weighted) voting to combine its base-learners predictions. This solution showed very promising results, exhibiting improved accuracy and robustness, and outperforming all single models. The single model Ictu Oculi achieved the overall highest accuracy among the CNNs but still fails to reach the accuracy level of the proposed ensemble solution (83.77, 98.41%). Such results pave the way for many possible future works and are in line with research findings in related fields. Given these outcomes, ensemble learning will likely play a big part in deepfakes detection, in the same way as they have in other fields of computer vision and image processing.

The contribution of this work lies in the following aspects: (1) This study is an attempt to introduce the concept of ensemble learning to deepfakes detection by evaluating four current detection methods and six different ensemble approaches. (2) It confirms that modern deepfakes are more difficult to detect for detection methods created for older deepfakes, demonstrating the necessity of ensemble learning for deepfakes detection. (3) By evaluating different ensemble approaches, the importance of choosing the most suitable approach becomes apparent, and in this case, it was to build an ensemble from all available single models and use soft (weighted) voting

when combining their predictions.

The source code used in the experiment is available on GitHub, as mentioned in the introduction of [Section 3](#).

7.1 Future work

Primarily, future work would concern incorporating other deepfakes detection methods to further map out the capabilities and performance of single deepfakes detection models and which affect the different single models would have on the ensemble. Such possibilities include extra experiments keeping the same or a similar experimental setup. Additionally, as I did not include individual preprocessing phases for every single model during re-training, adding these phases to see if they increase detection performance would increase the validity of these results.

The scope of this study concerned obtaining high deepfakes detection performance regardless of the actual inference time. *Inference time* refers to the process of using a trained ML algorithm to make a prediction on new data. As inference time is a key element in real-life applications and affects both resource utilization and power consumption, if I had continued working on this project I would make an analysis of important evaluation metrics for real-life applications and compare the ensembles to single models in terms of both accuracy and computational requirements related to actual deployment.

Also related to real-life applications, practitioners using these methods will request a justification for the numerical score for the analysis to be acceptable for publishing but due to the black-box nature of CNN models, deepfakes detection methods usually lack detailed explainability. Here it would be interesting to explore the area of explainable AI (XAI) and if a modified version of the proposed solution could be found where accuracy and comprehensibility could be combined.

Another aspect of real-life situations is how robust the proposed ensemble solution is towards anti-forensic methods and social media laundering. Regarding the first aspect, anti-forensic measures are developed to prevent detection models from being able to distinguish between real and fake videos by adding simulated signal level features. The proposed detection solution must be improved to handle such intentional and adversarial attacks. Regarding the second aspect, social media laundering refers to the fact that videos are down-sized and heavily compressed before being uploaded to social media platforms. How affected would the proposed solution be by this? If greatly affected, this might profoundly impact the detection performance, especially the recovery traces of manipulated videos and increasing the false positive (FP) detections. A practical measure to solve this would be to incorporate simulations of such effects in the training data and enhance the test set used for evaluation to include performance on social

media laundered videos, both real and fake ones.

Lastly, as mentioned in Section 1.6, the scope of this study only included one type of fake videos, those produced by deepfake technology. So, evolving the proposed detection solution to include all forms of forged images and videos, as well as audio deepfakes, would be an appropriate way to continue this research. If it turns out that different methods are needed to specifically target the different forgeries, could those methods be combined into an ensemble model to create a complete detection system?

References

- [1] I. Goodfellow, Y. Bengio and A. Courville. *Deep Learning*. Cambridge, MA: MIT Press, 2016.
- [2] S. Lyu, "DeepFake Detection: Current Challenges and Next Steps", arXiv preprint arXiv:2003.09234, 2020.
- [3] H. Hasan and K. Salah, "Combating deepfake videos using blockchain and smart contracts," *IEEE Access*, vol. 7, no. 1, pp. 41596–41606, Dec. 2019.
- [4] A. Qayyum, J. Qadir, M. U. Janjua, and F. Sher, "Using Blockchain to Rein in The New Post-Truth World and Check The Spread of FakeNews", arXiv preprint arXiv:1903.11899, 2019.
- [5] H. H. Nguyen, J. Yamagishi, and I. Echizen, "Use of a Capsule Network to Detect Fake Images and Videos" arXiv preprint arXiv:1910.12467, 2019.
- [6] Y. Li and S. Lyu, "Exposing DeepFake Videos By Detecting Face Warping Artifacts", arXiv preprint arXiv:1811.00656, 2019.
- [7] Y. Li, M-C. Chang and S. Lyu, "In Ictu Oculi: Exposing AI Generated Fake Face Videos by Detecting Eye Blinking", arXiv preprint arXiv:1806.02877, 2018.
- [8] A. Rössler, D. Cozzolino, L. Verdoliva, C. Riess, J. Thies, M. Nießner, "FaceForensics++: Learning to Detect Manipulated Facial Images", arXiv preprint arXiv:1901.08971, 2019.
- [9] A. Khan, A. Sohail, U. Zahoora, and A. S. Qureshi, "A Survey of the Recent Architectures of Deep Convolutional Neural Networks", arXiv preprint arXiv:1901.06032, 2020.
- [10] L. Verdoliva, "Media Forensics and DeepFakes: an overview", arXiv preprint arXiv:2001.06564, 2020.

- [11] F. Perez, S. Avila, E. Valle, "Solo or Ensemble? Choosing a CNN Architecture for Melanoma Classification", arXiv preprint arXiv:1904.12724, 2019.
- [12] T.D. Fikse, "Imagining Deceptive Deepfakes: An ethnographic exploration of fake videos", Master's thesis, ESST – Society, Science and Technology in Europe, University of Oslo, Oslo, Norway, 2018.
- [13] Kaggle. (2019) GAN Introduction. [Online]. Available: <https://www.kaggle.com/jesucristo/gan-introduction>
- [14] Y. Li, X. Yang, P. Sun, H. Qi, S. Lyu, "Celeb-DF (v2): A New Dataset for DeepFake Forensics" arXiv preprint arXiv:1909.12962, 2020.
- [15] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*, Cambridge University Press, 2008.
- [16] J. A. Anderson. *An Introduction to Neural Networks*. Massachusetts Institute of Technology, 1997.
- [17] N. S. Keskar, D. Mudigere, J. Nocedal, M. Smelyanskiy, and P. T. P. Tang, "On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima" arXiv preprint arXiv:1609.04836, 2017.
- [18] L. N. Smith, "Cyclical Learning Rates for Training Neural Networks" arXiv preprint arXiv:1506.01186, 2017.
- [19] R. C. Gonzalez, R. E. Woods, S. L. Eddin. *Digital Image Processing*. Pearson, 2007.
- [20] J. Bouvrie, "Notes on Convolutional Neural Networks", Center for Biological and Computational Learning, Department of Brain and Cognitive Sciences, Massachusetts Institute of Technology, Cambridge, MA, 2006.
- [21] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov. "Dropout: a simple way to prevent neural networks from

overfitting.” 2014 *J. Mach. Learn. Res.* 15, 1929–1958.

[22] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition” arXiv preprint arXiv:1409.1556, 2015.

[23] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition”, arXiv preprint arXiv:1512.03385, 2015.

[24] F. Chollet, “Xception: Deep learning with depthwise separable convolutions,” in *Proc. CVPR*, 2017.

[25] S. Tao, “Deep Neural Network Ensembles”, arXiv preprint arXiv:1904.05488, 2019.

[26] L. K. Hansen and P. Salamon, “Neural Network Ensembles”, *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 12(10):993 - 1001, Nov. 1990.

[27] Z.H. Zhou. *Ensemble Methods: Foundations and Algorithms*. Boca Raton, FL: Taylor & Francis Group, LLC, 2012.

[28] Z.H. Zhou, J. Wu, and W. Tang. “Ensembling neural networks: Many could be better than all”, Dec. 2010 *Artificial Intelligence*. Volume 174, Issue 18, 1570.

[29] A. Zheng. *Evaluating Machine Learning Models*. O'Reilly Media, Inc., 2015.

[30] M. Hossin & S. M.N. “A Review on Evaluation Metrics for Data Classification Evaluations.” 2015 *International Journal of Data Mining & Knowledge Management Process*. 5. 01-11. 10.5121/ijdkp.2015.5201.

[31] Charles Sturt University. (2020) Literature Review: Developing a search strategy. [Online]. Available: <https://libguides.csu.edu.au/review>

[32] N. Rijal, R. T. Gutta, T. Cao, J. Lin, Q. Bo, and J. Zhang, " Ensemble of Deep Neural Networks for Estimating Particulate Matter from Images", 2018 *IEEE 3rd International Conference on Image, Vision and Computing (ICIVC)*, Chongqing, 2018, pp. 733-738.

- [33] Z. He and S. Yang. “Multi-view Commercial Hotness Prediction Using Context-aware Neural Network Ensemble”, Dec. 2018 *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.* 2, 4, Article 168.
- [34] N. Bonettini, E. D. Cannas, S. Mandelli, L. Bondi, P. Bestagini, and S. Tubaro, “Video Face Manipulation Detection Through Ensemble of CNNs”, arXiv preprint arXiv:2004.07676, 2020.
- [35] S. Sabour, N. Frosst and G. E Hinton, “Dynamic Routing Between Capsules” arXiv preprint arXiv:1710.09829, 2017.
- [36] K. He, X. Zhang, S. Ren, and J. Sun, “Spatial Pyramid Pooling in Deep Convolutional Networks for Visual Recognition”, arXiv preprint arXiv:1406.4729, 2015.
- [37] D. K. Citron, R. Chesney. (2019) Deep Fakes: A Looming Challenge for Privacy, Democracy, and National Security, 107 California Law Review 1753. [Online]. Available:
https://scholarship.law.bu.edu/faculty_scholarship/640
- [38] V. Schetinger, M.M. Oliveira, R. da Silva, T. Carvalho, “Humans Are Easily Fooled by Digital Images”, arXiv preprint arXiv:11509.05301, 2015.
- [39] P. Langley and D. Kibler. *The Experimental Study of Machine Learning*. Cambridge, 1997.
- [40] GitHub. (2019) Celeb-DF: A Large-scale Challenging Dataset for DeepFake Forensics. [Online]. Available:
<https://github.com/danmohaha/celeb-deepfakeforensics>
- [41] Google AI Blog. (2019) Contributing Data to Deepfake Detection Research. [Online]. Available:
<https://ai.googleblog.com/2019/09/contributing-data-to-deepfake-detection.html>
- [42] Kaggle. (2020) Deepfake Detection Challenge Training Set. [Online]. Available: <https://www.kaggle.com/c/deepfake-detection-challenge/data>

- [43] Lagen.nu. (2018) Lag (1960:729) om upphovsrätt till litterära och konstnärliga verk. [Online]. Available: <https://lagen.nu/1960:729>
- [44] GDPR. (2018) Chapter III Rights of the data subject Article 17. Right to erasure ('right to be forgotten'). [Online]. Available: <https://gdpr.algolia.com/gdpr-article-17>
- [45] GitHub. deepfakes/faceswap. (2020). Manifesto: FaceSwap has ethical uses. [Online]. Available: <https://github.com/deepfakes/faceswap#manifesto>
- [46] PyTorch. Matthew Inkawhich. (2017). Saving and Loading Models. [Online]. Available: https://pytorch.org/tutorials/beginner/saving_loading_models.html